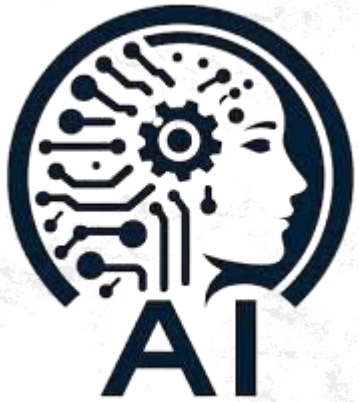


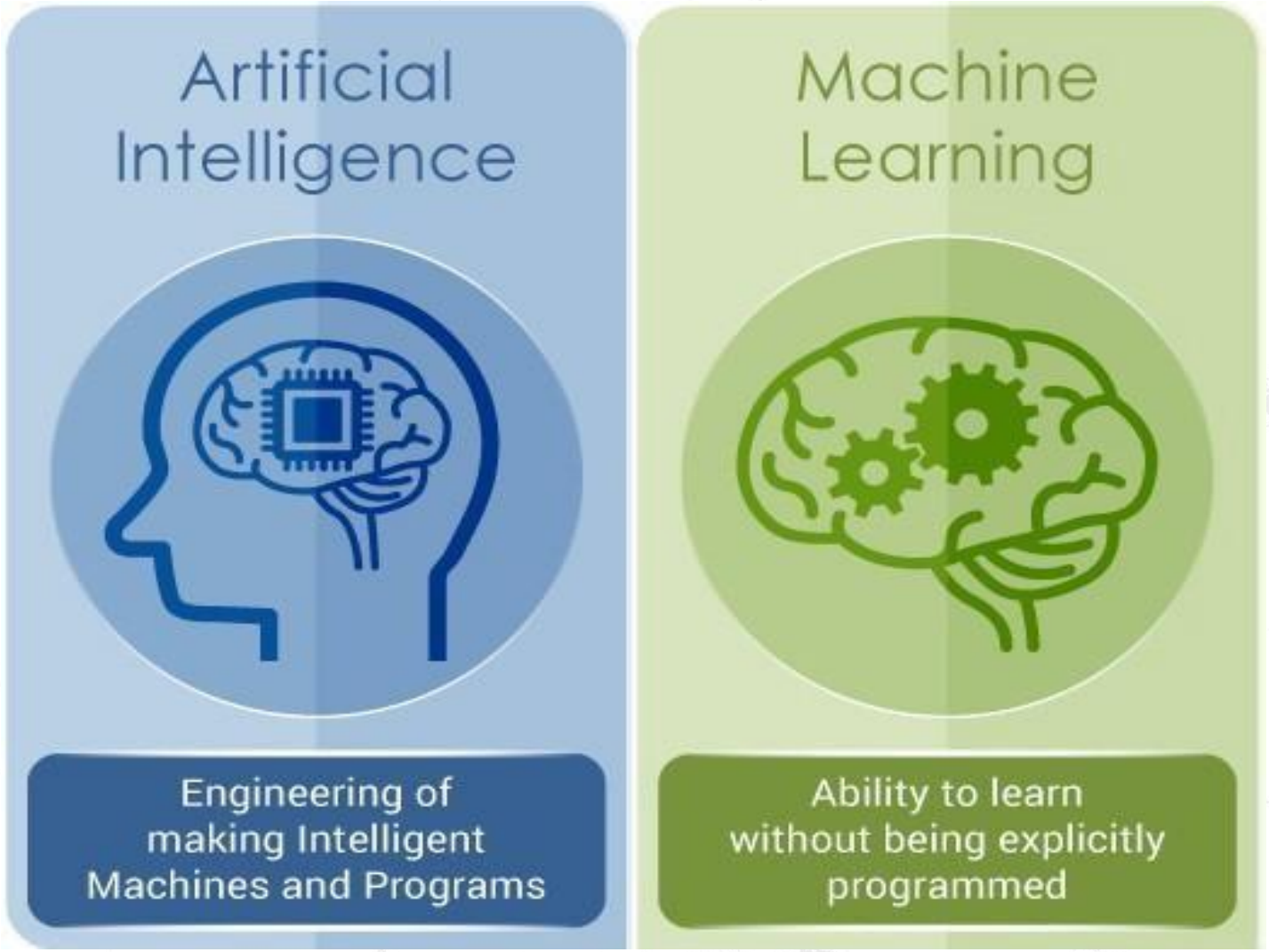
Overview of Python as an Artificial Intelligence (AI/ML) language

Introduction to Python

- Python is a popular programming language. It was created by Guido van Rossum, and released in 1991.
- Python is a versatile and beginner-friendly programming language that has gained immense popularity in Data Science and Machine Learning due to its simplicity and a rich ecosystem of libraries.

It is used for:
Data Science and AIML
Web development (server-side),
Software development,
Mathematics,
System scripting.

OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE



OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

AI- artificial intelligence:

1. It is the science of training machines to perform human tasks
2. The term was invented in the 1950s when scientists began exploring how computers could solve problems on their own.
3. AI is a computer that is given human-like properties. Like our brain which works effortlessly and seamlessly to calculate the world around us.
4. Artificial Intelligence is the concept that a computer can do the same.
5. It can be said that AI is the large science that mimics human aptitudes.

Machine learning:

1. It is a subset of AI that trains a machine how to learn
2. It is the art of study of algorithms that learn from examples and experiences.
3. Machine learning is based on the idea that there exist some patterns in the data which are to be identified and used for future predictions.
4. The machine does not need to be explicitly programmed by people. The programmers give some examples, and the computer is going to learn what to do from those samples.
5. The difference from hardcoding rules is that the machine learns on its own to find such rules.

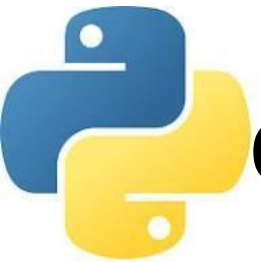
OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

Why Python is Preferred for AI/ML?

Python has become the go-to programming language for **artificial intelligence** (AI) development due to its **simplicity** and the **powerful suite of libraries** it offers

Its **syntax is straightforward** and closely **resembles human language**, which reduces the learning curve for developers and enables them to focus on solving AI problems rather than wrestling with complex coding issues .Python is highly favored for AI and machine learning (ML) development for several compelling reasons that make it uniquely suitable for these technologies:

1). **Extensive Libraries and Frameworks**: Python boasts a wide range of libraries and frameworks, such as Scikit-learn for machine learning algorithms, TensorFlow, PyTorch, and Keras for deep learning, as well as NumPy, Pandas, and Seaborn for data manipulation and visualization. These libraries simplify coding tasks and reduce development time by providing pre-written code for common tasks.



OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

2) Ease of Learning and Syntax Simplicity: Python's syntax is straightforward, resembling everyday English, which significantly lowers the learning curve and enables developers to implement complex AI algorithms efficiently.
The simplicity of Python syntax, which avoids brackets and emphasizes indentation, contributes to its ease of use and readability, making code less prone to errors and more maintainable .

3) No Need to Recompile Source Code: Python allows for dynamic modification and execution of code without the need for recompilation, offering flexibility and speeding up the development process. This feature is particularly advantageous in AI and ML projects, where iterative testing and tweaking are common.

OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

4) Platform Independence: Python code can run on various operating systems, including Windows, Mac, UNIX, and Linux, without requiring any modifications, making it highly versatile for development across different platforms .

5) Strong Community Support: Python's large and active community contributes to a wealth of resources, including tutorials, forums, and documentation, which are invaluable for developers' encountering challenges or seeking to enhance their knowledge and skills in AI and ML.

OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

Artificial Intelligence (AI) is a discipline that focuses on creating intelligent machines that can perform tasks that typically require human intelligence, such as visual perception, speech recognition, decision-making, and natural language processing. It involves the development of algorithms and systems that can reason, learn, and make decisions based on input data.

Essential Python Libraries for AI/ML

For anyone diving into Artificial Intelligence (AI) using Python, a handful of libraries make the development process significantly smoother and more efficient. These libraries, equipped with pre-built functions and tools, are essentials in the AI developer's toolkit:

NumPy: Fundamental for scientific computing in Python, NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more. It's particularly useful for handling large, multi-dimensional arrays and matrices, which are prevalent in AI tasks.

Pandas: A library offering high-level data structures and tools designed to make data analysis fast and easy in Python. Pandas are ideal for data munging and preparation and can be used in conjunction with other data analysis workflows in Python.

OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

Matplotlib: This plotting library allows for the creation of static, animated, and interactive visualizations in Python. Visual data representation is crucial in AI for data exploration and the presentation of results.

TensorFlow: An open-source library developed by the Google Brain team, TensorFlow is used for numerical computation using data flow graphs. It's particularly known for its applications in deep learning and neural networks.

PyTorch: Developed by Facebook's AI Research lab, PyTorch is a library for machine learning that provides great flexibility and speed while working with deep learning projects. It's known for its ease of use and efficiency in creating complex AI models.

Machine Learning Introduction

Agenda

- Introduction ,History
- Why ML is Important (ExamplesWeather Forcast,Face Recogination,Attendence sys)
- ML vs Traditional Programing
- How does Machine Learning Works
- What is Data , Dataset , Pattern making , Prediction
- Activity
- Types Of ML(Supervised and Unsupervused)
- Linear Regression Working
- Supervised Vs Unsupervused
- Activity : Teachable machine,Diabetes Prediction

Introduction

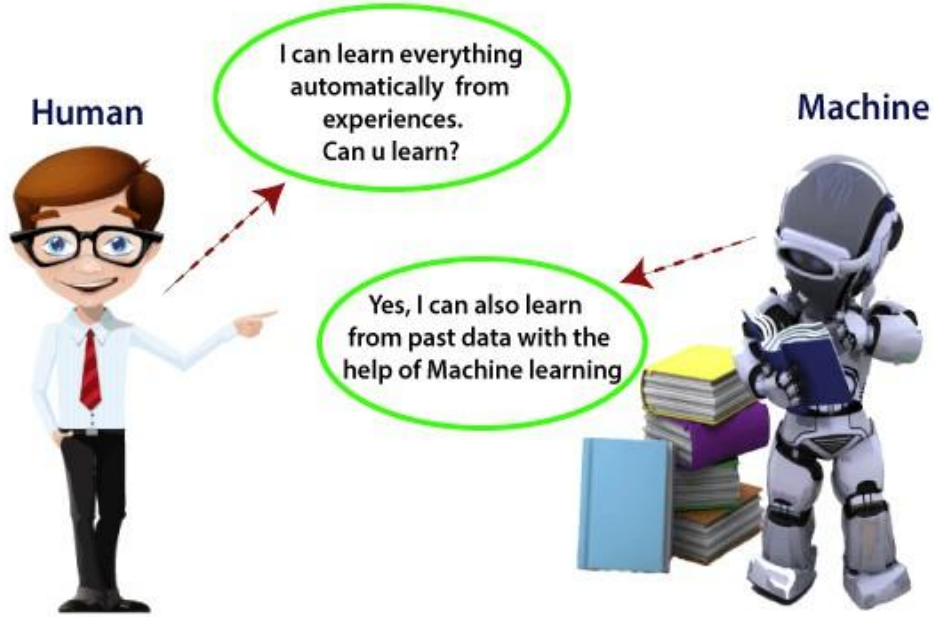
Machine Learning is when computers learn to do things on their own without being told exactly what to do every time.

Example :

Think of it like teaching your friend how to play a game—once they understand the rules, they get better with practice.

Introduction

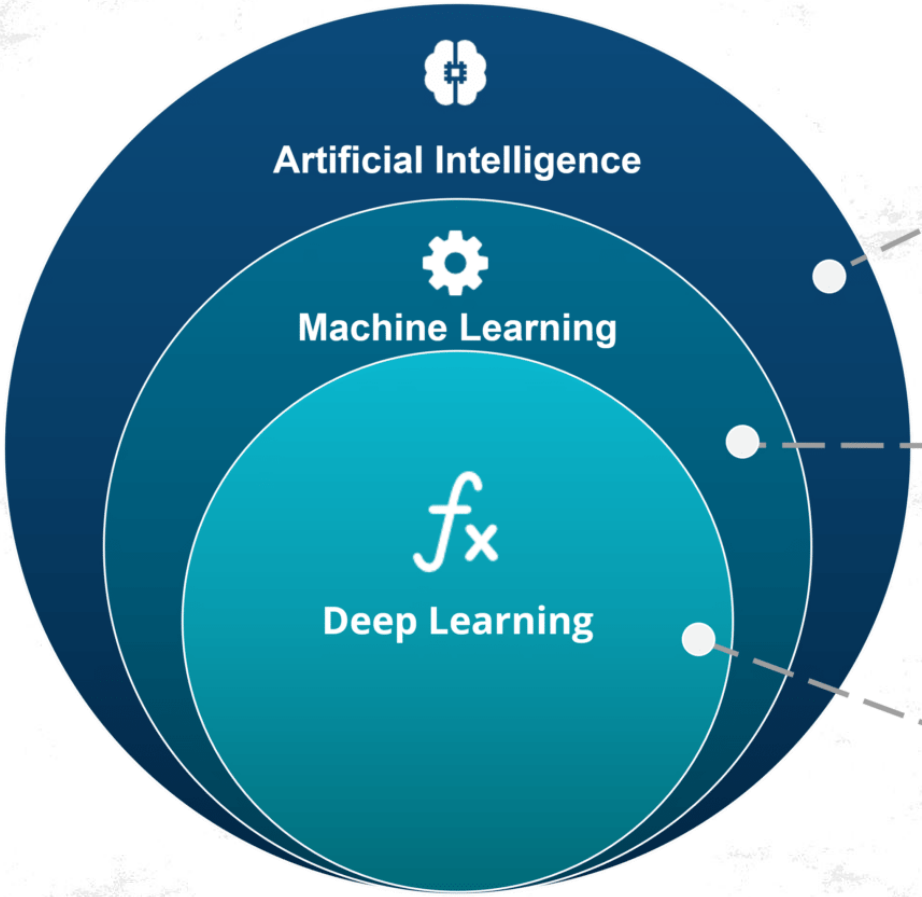
- In the real world, we are surrounded by humans who can learn everything from their experiences with their learning capability, and we have computers or machines which work on our instructions.



So here comes the role of

Machine Learning.

Introduction



ARTIFICIAL INTELLIGENCE
A technique which enables machines to mimic human behaviour

MACHINE LEARNING
Subset of AI technique which use statistical methods to enable machines to improve with experience

DEEP LEARNING
Subset of ML which make the computation of multi-layer neural network feasible

Example

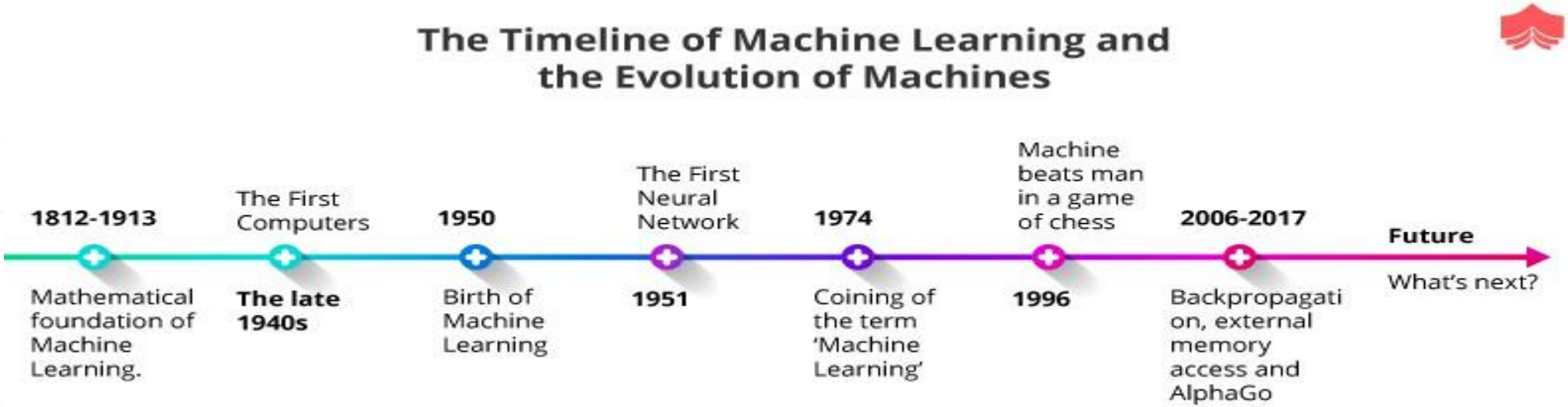
Imagine you are training your pet dog.

- If you say "sit" and give the dog a **treat** every time it sits, the dog learns that sitting gets a reward.
- Over time, the dog learns to sit whenever you say "sit," even if you don’t give a treat.

In the same way, computers use data (the treats) to learn patterns and make decisions.

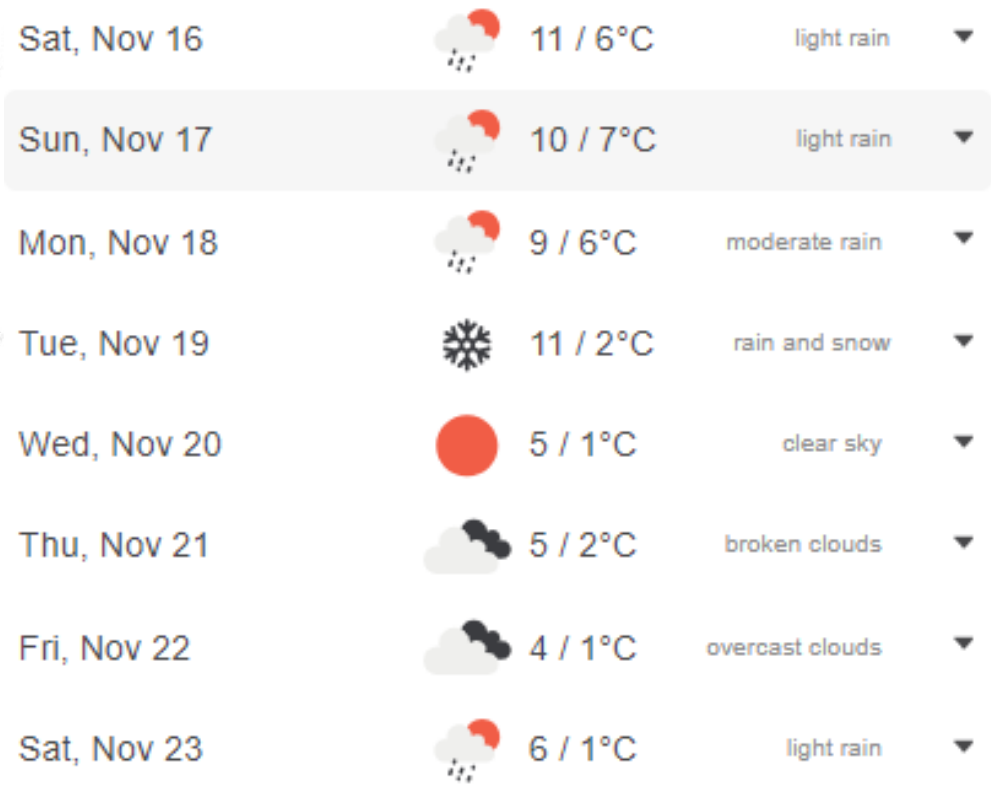
History

- Before some years (about 40-50 years), machine learning was science fiction, but today it is the part of our daily life. Machine learning is making our day to day life easy from **self-driving cars** to **Amazon virtual assistant "Alexa"**.



Why is Machine Learning Important?

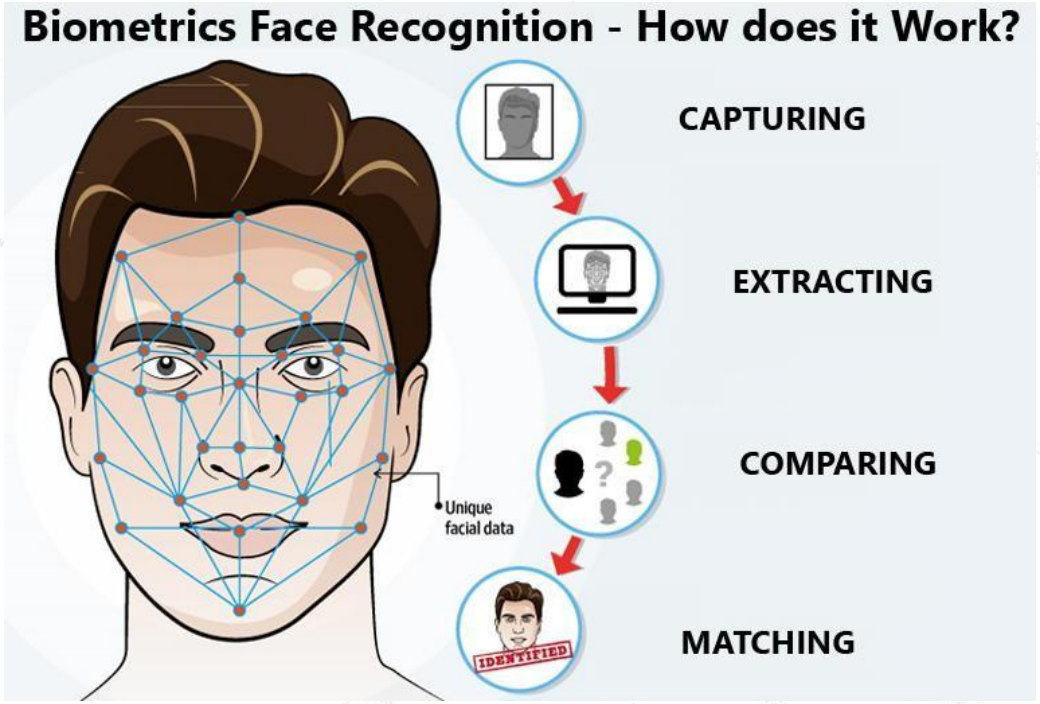
1. **Predicting Things (e.g., Weather Forecasts)**
   - We check the weather every day to decide what to wear or carry (like an umbrella).
   - Machine Learning helps us know what might happen in the future by looking at past data.
   - This helps us plan our day better!



Why is Machine Learning Important?

Recognizing Faces or Voices (e.g., Unlocking Your Phone)

- Your phone "learns" how your face looks by studying pictures of it.
- When you try to unlock the phone, it matches your face to what it has learned.
- If the match is correct, the phone unlocks.

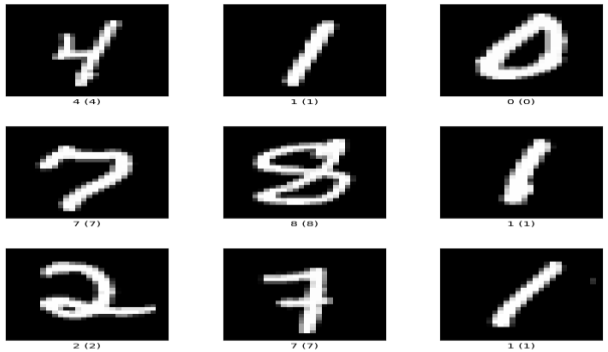


Example of Machine Learning Task: Recognizing Handwriting

- **Task (T):** The computer's job is to recognize and classify handwritten words in pictures.
  Example: It looks at an image and says, “This is the word *Hello*.”
- **Performance Measure (P):**
  We check how well the computer did its job. For handwriting, this means checking the **percentage of words it got correct**.
  If it recognizes 9 out of 10 words correctly, its performance is 90%.
- **Training Experience (E):**
  The computer learns by practicing with a **database of handwritten words** (lots of examples).
  It sees different handwriting styles and learns to recognize them.

Activity

https://www.ccom.ucsd.edu/~cdeotte/programs/MNIST.html



Machine Learning vs. Traditional Programming

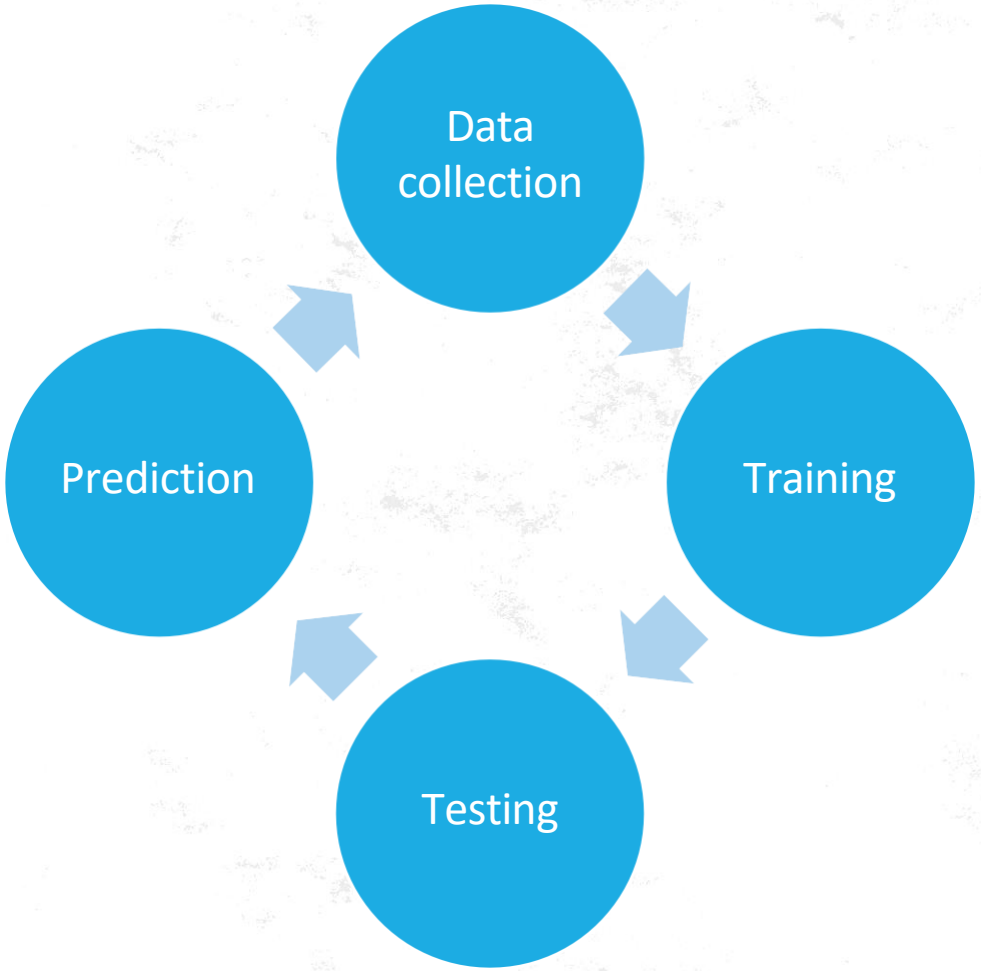
Traditional Programming

- **Rules-Based**: Relies on explicit instructions written by programmers.
- **Logic Driven**: Follows predefined logic and algorithms.
- **Input & Rules → Output**: Given input and a set of rules, produces a fixed output.

Machine Learning

- **Data-Driven**: Learns patterns from large data sets.
- **Model Training**: Requires training data to "learn" how to make predictions.
- **Input & Output** → Rules: Uses input-output pairs to infer rules (model).

How does Machine Learning work?



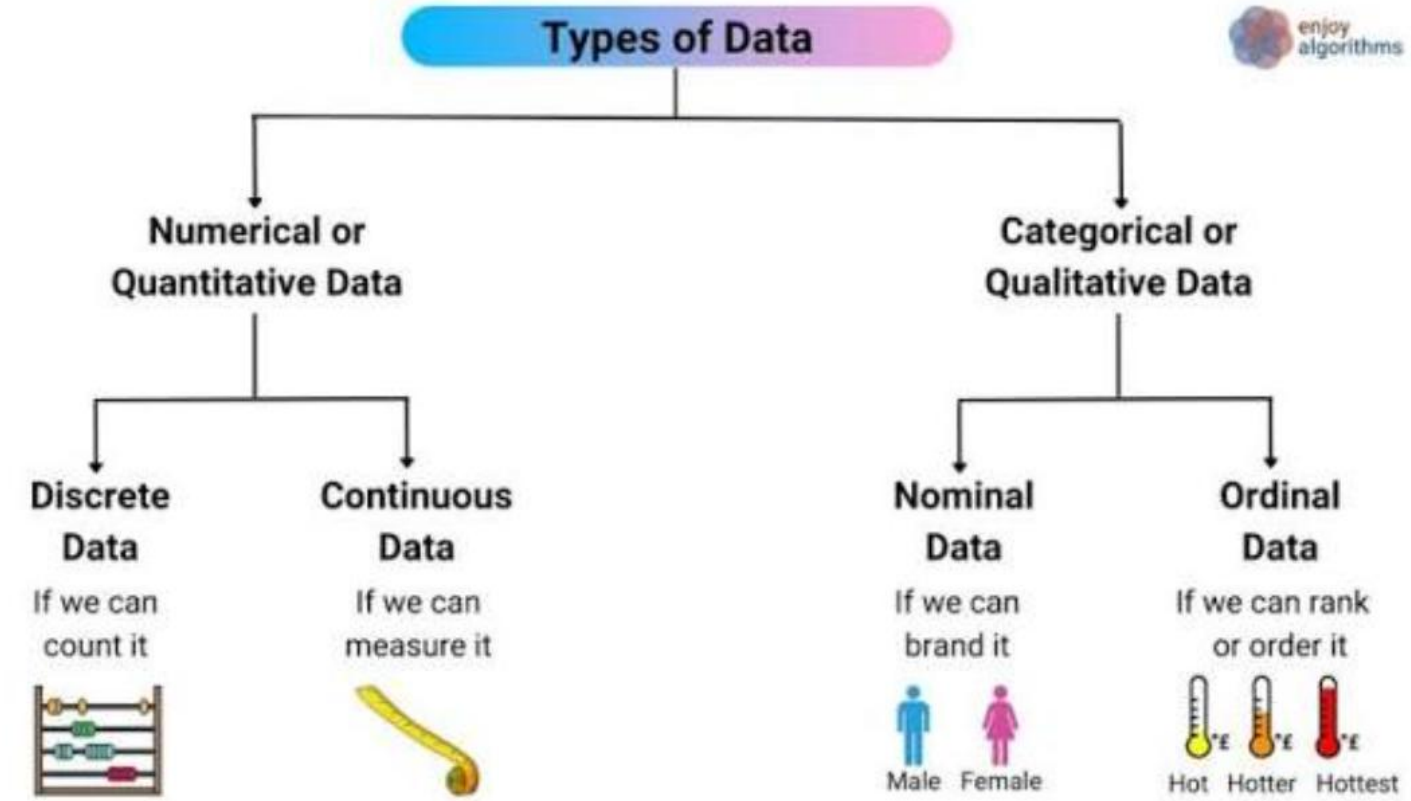
What is Data ?

- Data is information used by ML to learn.

Example

- Imagine you want to teach a computer to recognize fruits.
- You give it pictures of apples, bananas, and oranges (this is the **data**).
- Each picture is labeled so the computer knows which one is an apple, banana, or orange.
- The computer studies these pictures to learn the differences between the fruits, like their shape and color.

Types of Data

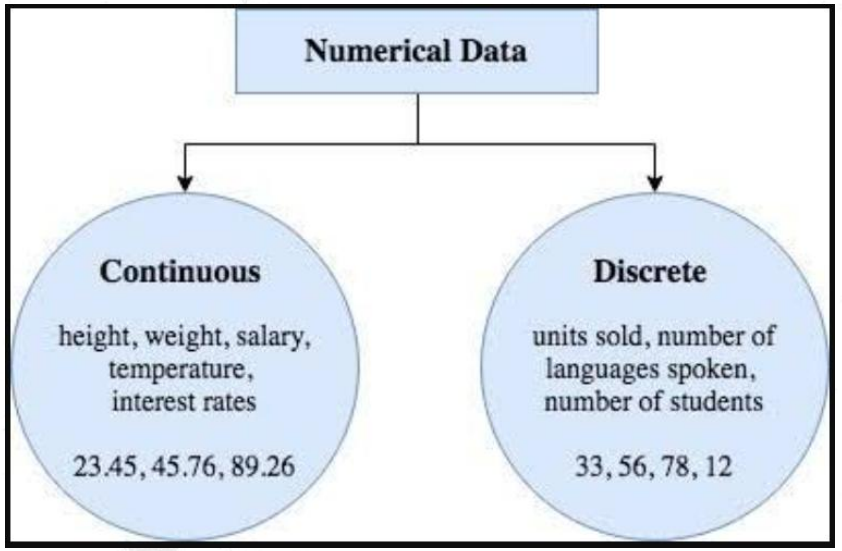


However, from a practical engineering perspective, data is also classified by its structural organization (Structured, Unstructured, Semi-Structured).

Numerical (Quantitative) Data

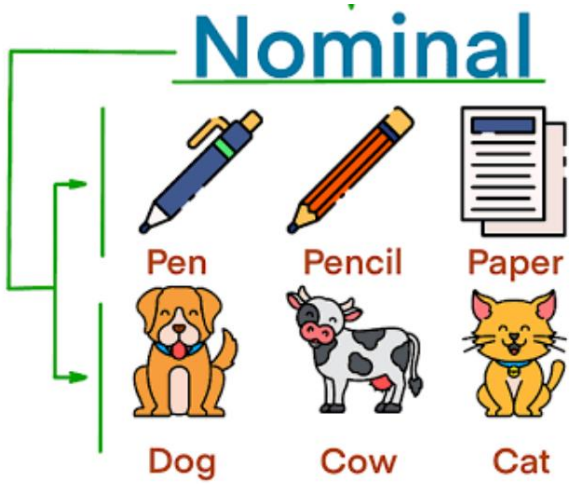
Consists of measurable, countable values expressed in numbers.

- **Continuous Data**: Values that can fall anywhere within a given range, including decimals. Examples include weight, temperature, and stock market indices.
- **Discrete Data**: Countable, distinct whole numbers that cannot be broken down into fractions. Examples include the number of children in a family or website clicks.



Categorical (Qualitative) Data

Represents characteristics, distinct groups, or non-numeric labels.



- **Nominal Data:** Categories that have no inherent ranking, order, or quantitative value. Examples include gender, color, or country of residence.

- **Ordinal Data:** Categorical data that follows a meaningful, sequential order or ranking system. Examples include customer feedback ratings (e.g., Poor, Fair, Good, Excellent) or clothing sizes (S, M, L).

Example of Dataset

Name	English Marks	Favourite Colour	Sports
Aditi	85	Blue	Football
Raj	78	Green	Cricket
Simran	92	Red	Badminton
Aarav	60	Blue	Cricket
Pooja	70	Yellow	Football
Kabir	88	Red	Badminton

Patterns In Dataset

- Finding Patterns in Marks

  Who scored the highest marks in English?

- Finding Patterns in Favourite Colour

  Which colour is most popular? Which sport is played by the most students?

- Finding Patterns in Sports
- Deeper Patterns

  Can you spot any connection between favourite colour and sports?

Name	Favourite Color	Sports
Simran	Red	Badminton
Kabir	Red	Badminton

Activity No.1

- 1. **Do you notice any connections between English marks and sports?**

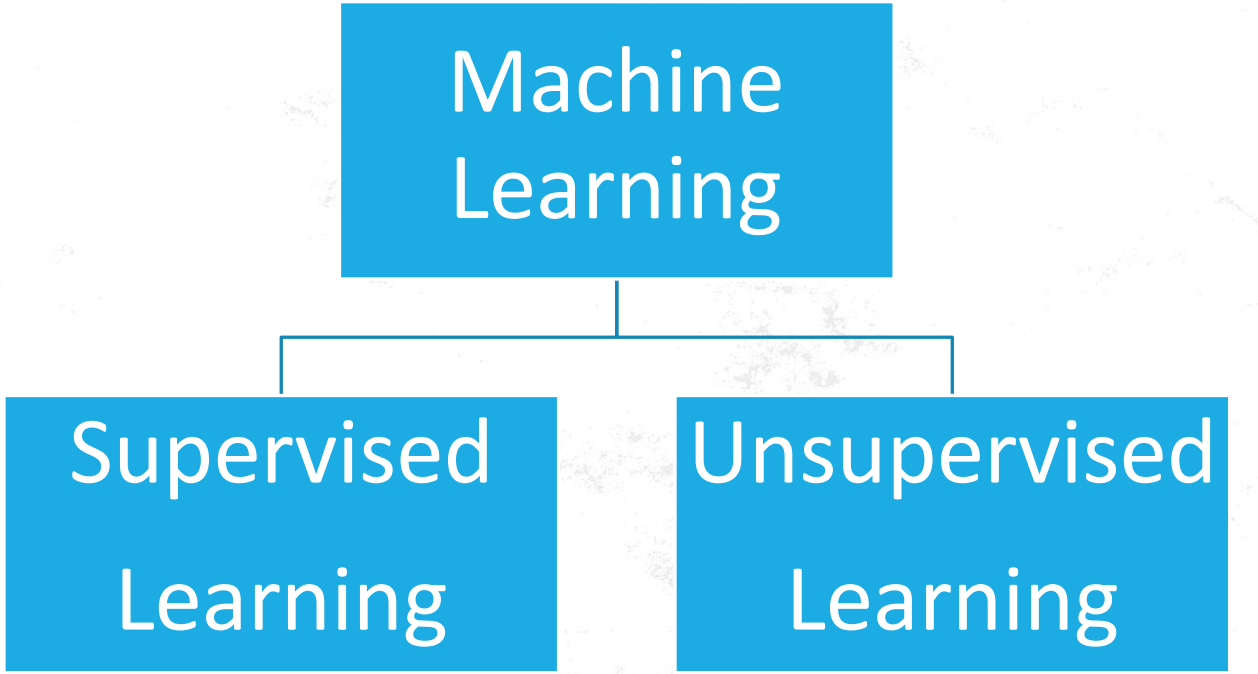
  Observation:

  1. Students with higher marks (above 85, like Simran and Kabir) are associated with Badminton.
  2. Students with moderate marks (70-85, like Aditi, Raj, and Pooja) are associated with Football and Cricket.
  3. Students with lower marks (like Aarav) tend to play Cricket.

Activity No.2

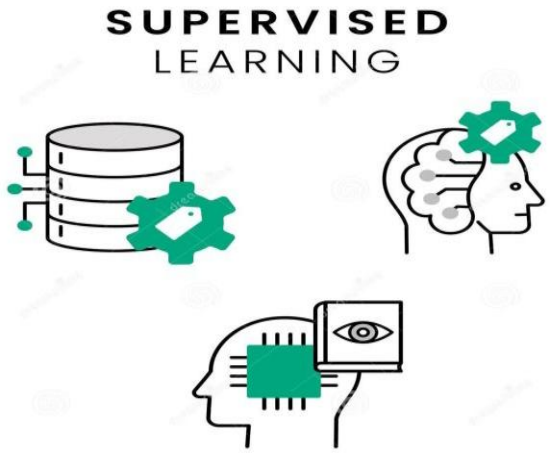
- Can you predict what sport or favourite colour a new student might like based on the existing data?
  - A student scores greater than 85 in English and their favourite colour is **Red**.
    → Likely sport: **Badminton**.

Classification of Machine Learning



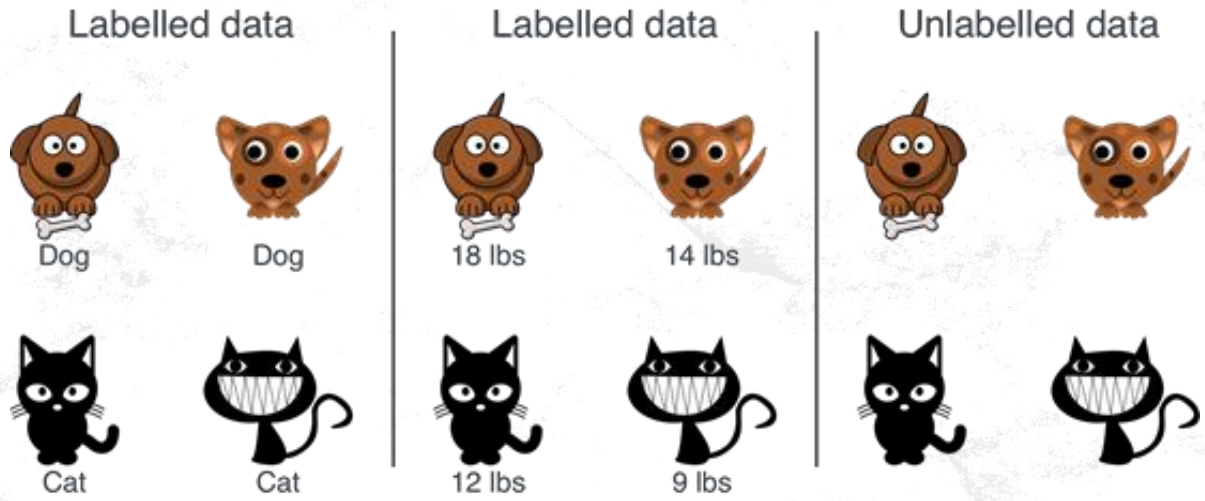
Supervised learning

- Supervised machine learning learns patterns and relationships between input and output data. It is defined by its use of labeled data.
- The training process continues until the model achieves a desired level of accuracy on the training data. once this model is determined, it can be used to apply labels to new, unknown data



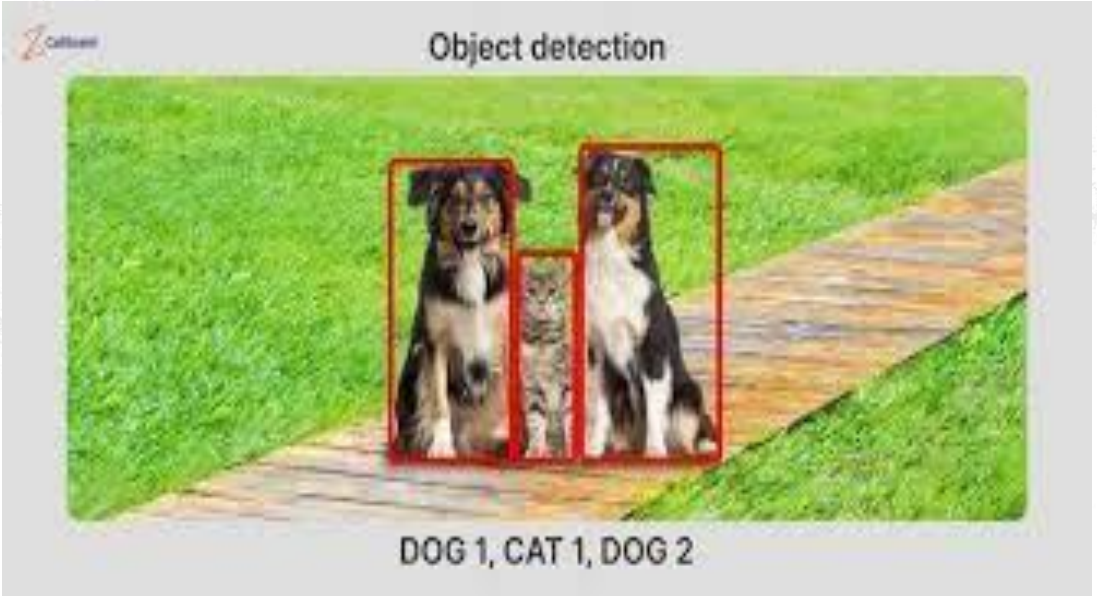
Supervised learning

- **Labeled Data :**Labeled data is data that has both inputs and correct answers. For example, an image labeled "dog" or "cat" tells a model what the picture shows. The labels help the model learn to make accurate predictions.



Supervised Learning

- **You give the computer a lot of examples where you also tell it the correct answer.**
  - **Show a picture of a dog and label it “Dog.”**
  - **Show a picture of a cat and label it “Cat.”**
- **Training the Model:**
  - The computer tries to find patterns in the examples.
  - It keeps practicing until it gets the answers mostly correct.



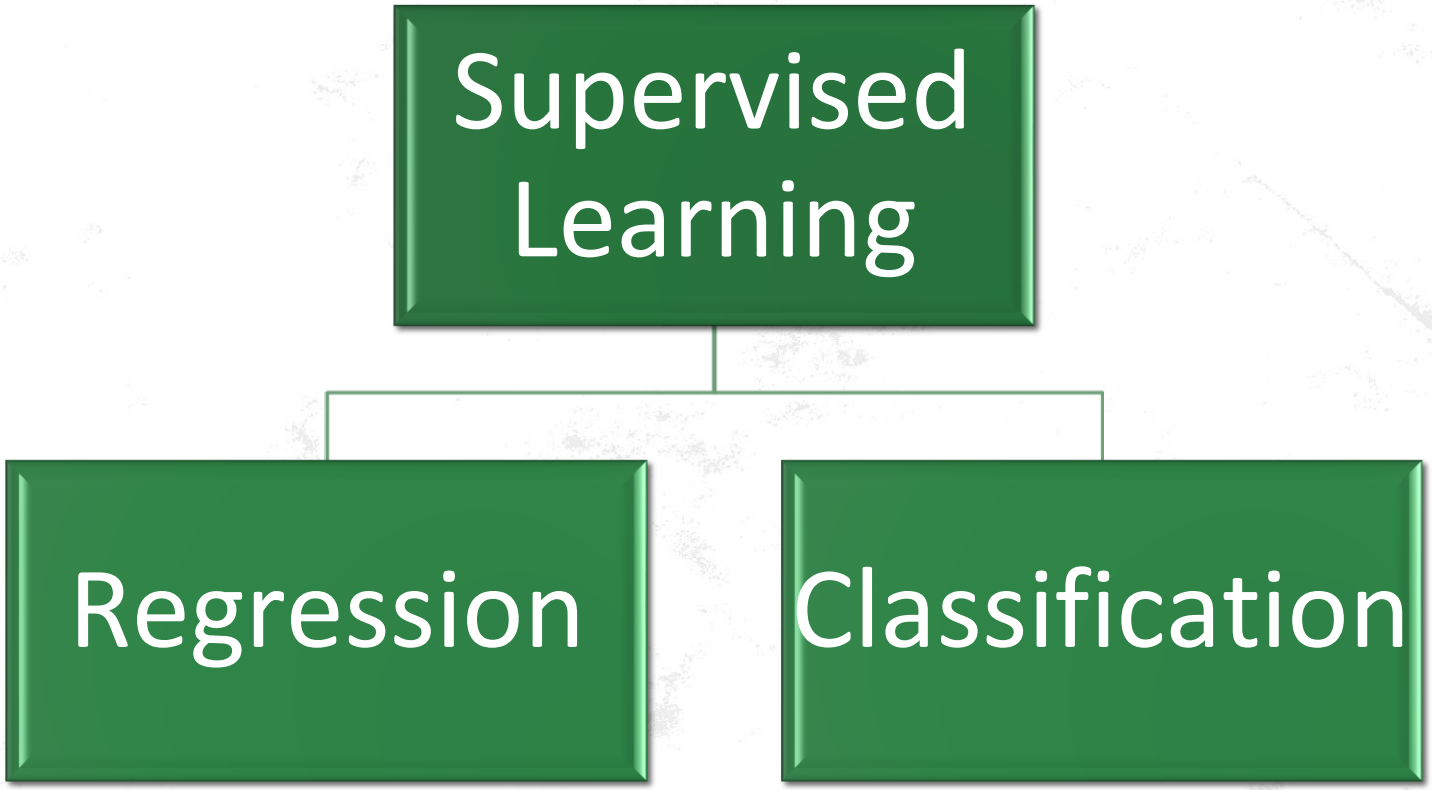
- **Using the Model:**

  Once the computer learns enough, you can show it new pictures (unknown data), and it will try to guess the correct label.
  For example: If you show it a new picture of a dog, it will say, “This is a Dog.”

Applications of Supervised learning

- Spam filtering
- Image classification
- Medical diagnosis
- Car Price Prediction
- Fraud detection
- Loan Price Prediction
- etc

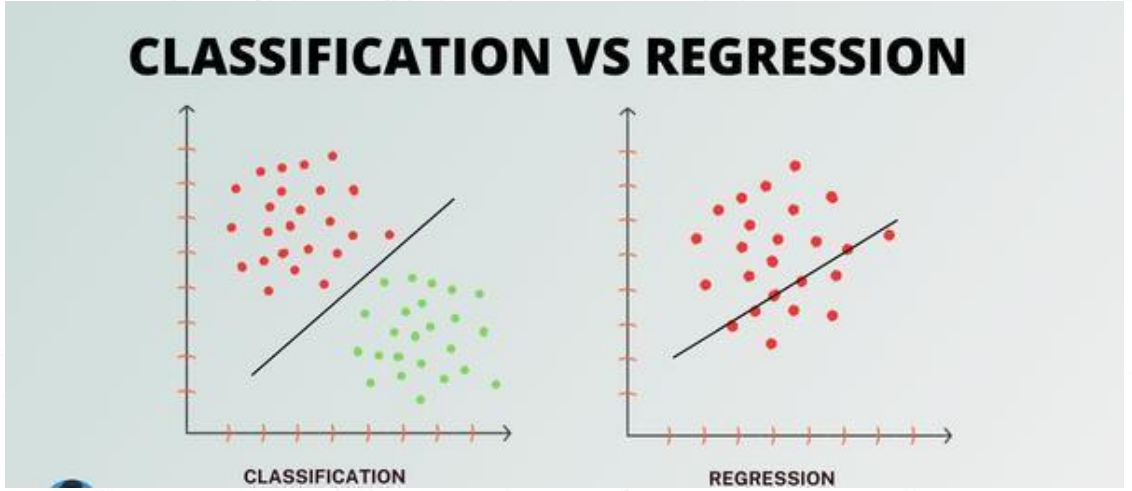
Types of Supervised Learning



Types of Supervised Learning

- **Classification**: A supervised learning task that assigns data to predefined categories, like labeling an email as "spam" or "not spam."

- **Regression**: A supervised learning task that predicts continuous numerical values, such as forecasting house prices based on property features.

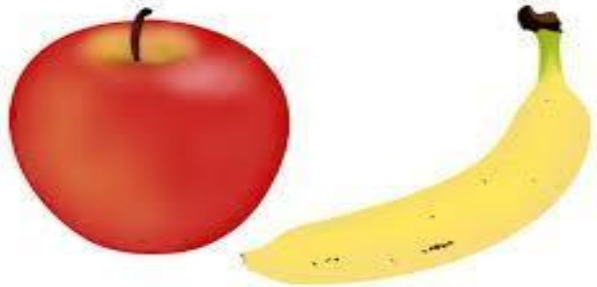


Classification

- Classification is like sorting things into groups or categories. The computer learns to put data into the right category.
- Imagine you have a basket of fruits. You sort the fruits into two groups:

  Apples

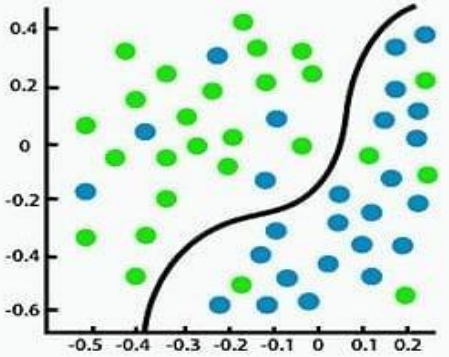
  Bananas



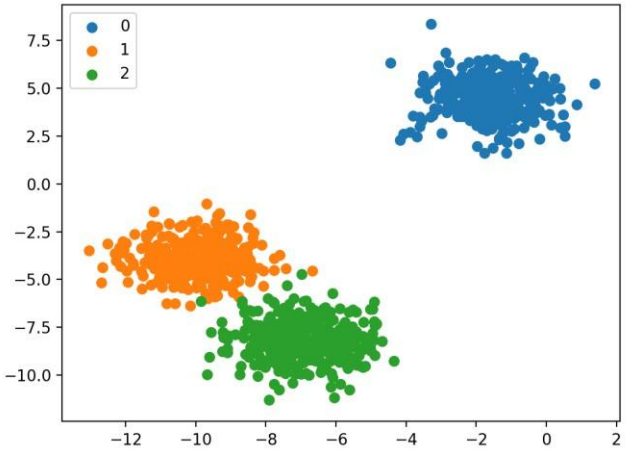
- Now, if you see a new fruit, you decide if it's an apple or a banana. That's what a computer does in **classification**!

Supervised Learning: Classification Problems

- Classification is about teaching a model to distinguish between different categories by learning from labeled examples.
- Each example in the training data belongs to a known class (like "cat" or "dog"), and the model uses this information to identify patterns in the features
- By finding boundaries that best separate these classes, the model can predict which class a new, unseen input belongs to. For example, in image recognition, classification helps the model decide whether an image shows a "cat," "dog," or another object based on features like shape and color.



Classification



Machine Learning steps

Preprocessing for Machine Learning model

1. Handling Missing Values

Machine learning models cannot handle blank spaces or NaN values. You must either remove them or fill them.

A) Drop rows with any missing values

```
df = df.dropna()
```

B) Fill missing numerical values with the column mean

```
df["age"] = df["age"].fillna(df["age"].mean())
```

Preprocessing for Machine Learning model

2. Removing Duplicates

Duplicate rows do not provide new information and can artificially bias your model's training process.

A) Remove identical rows, keeping the first occurrence

```
df = df.drop_duplicates()
```

Preprocessing for Machine Learning model

3. Encoding Categorical Data

Machine learning models only understand math, so text categories must be converted into numerical values.

A) Convert text columns (e.g., 'City', 'Color') into 0s and 1s

```
df = pd.get_dummies(
        df,
        columns=["city", "color"],
        drop_first=True
        )
```

Preprocessing for Machine Learning model

4. Managing Outliers

Extreme values (outliers) skew model performance. You can cap them at specific percentiles using capping/winsorization.

A) Cap values below the 1st percentile and above the 99th percentile

```
lower, upper =
df["income"].quantile([0.01, 0.99])

df["income"] =
df["income"].clip(lower=lower, upper=upper)
```

Preprocessing for Machine Learning model

5. Feature Scaling (Normalization/Standardization)

If features have wildly different ranges (e.g., Age 0–100 vs. Salary 0–1,000,000), distance-based models will fail.

A) Min-Max Scaling: Shifts all values to scale precisely between 0 and 1

```
df["salary"] =
  (df["salary"] - df["salary"].min()) /
  (df["salary"].max() - df["salary"].min())
```

Normalizing data = (x- xmin)/(xmax – xmin)

Preprocessing for Machine Learning model

6. Splitting Features and Targets

Before modeling, separate your independent variables (features, X) from your dependent variable (target, y).

A) X holds all columns except the target; y holds just the target column

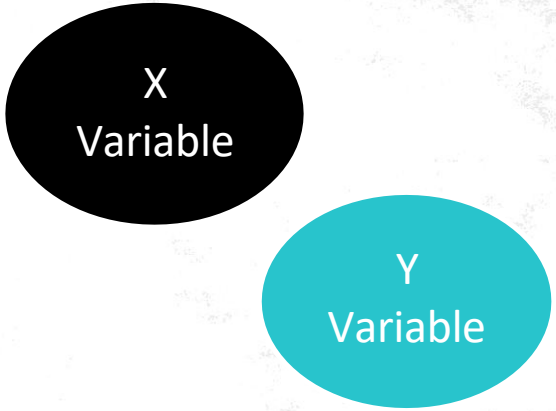
```
X = df.drop(columns=["target_column"])
y = df["target_column"]
```

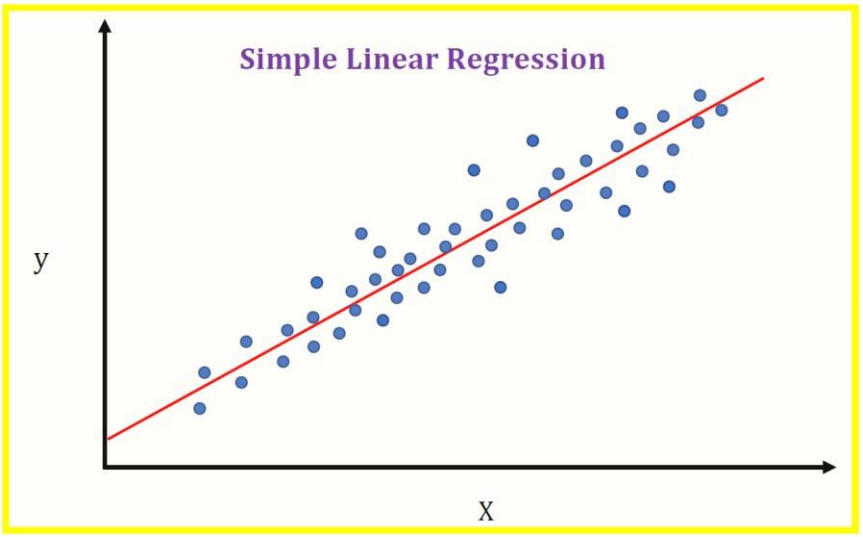
Regression

- Regression is a type of supervised machine learning where algorithms learn from the data to predict continuous values such as sales, salary, weight, or temperature.
- The goal of regression is to find the relationship between input variables (also called independent variables or features) and an output variable (also known as the dependent variable or target).
- This relationship allows the model to predict future values based on new data.

Regression

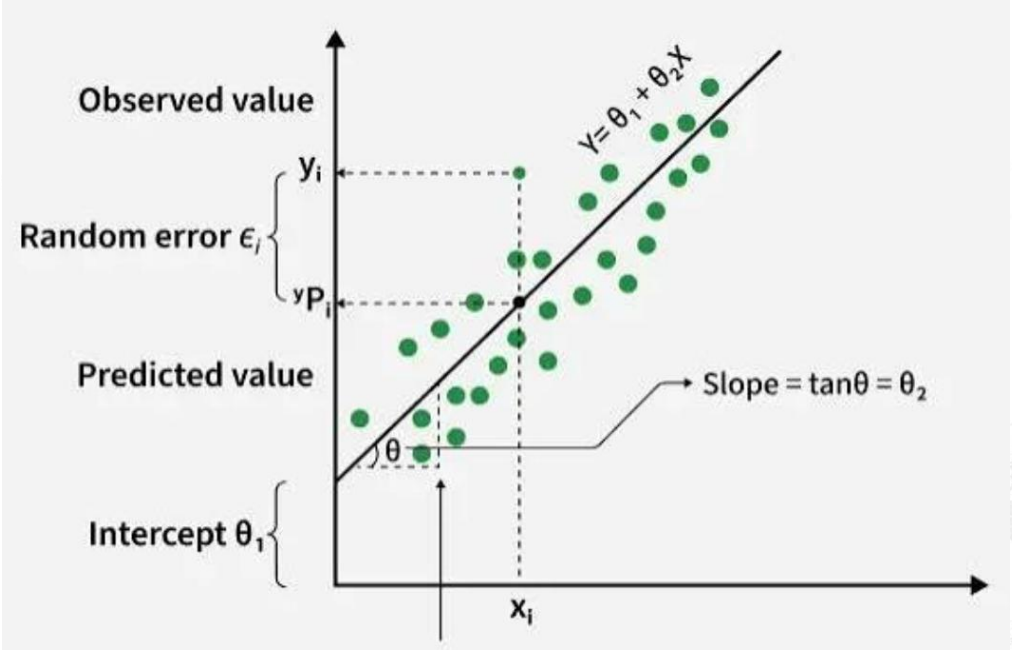
- The regression model analyzes historical data where both the input features and the output target are known. It tries to identify a mathematical function (often represented as a line or curve) that best represents the relationship between inputs and outputs. This function can then be applied to new input data to predict an output value.





1. The Intuition (The "Line of Best Fit")

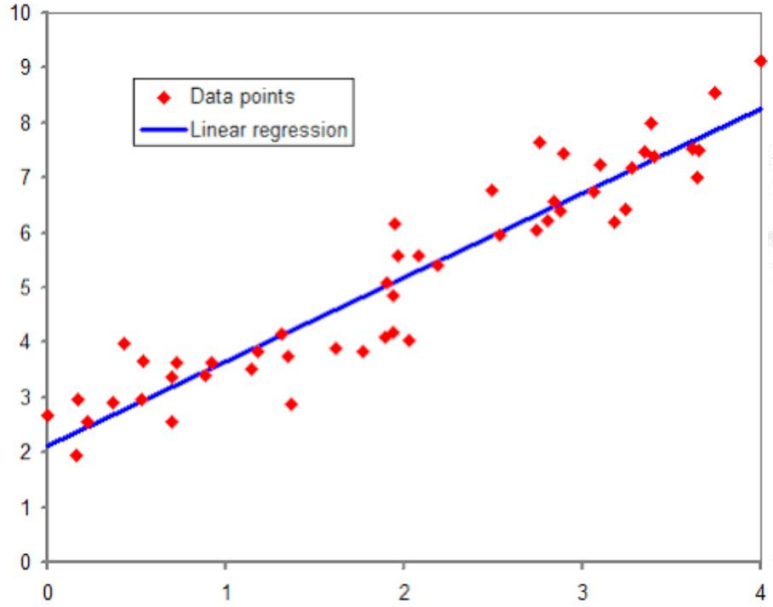
Plotting house sizes (x-axis) against their sale prices (y-axis). You get a scattered cloud of dots. Linear regression is simply the process of drawing a straight, angled line directly through the "center of mass" of that cloud to capture the overall trend.



The Core Equation

Every straight line in the universe is defined by one simple algebraic equation:

$$y = mx + b$$

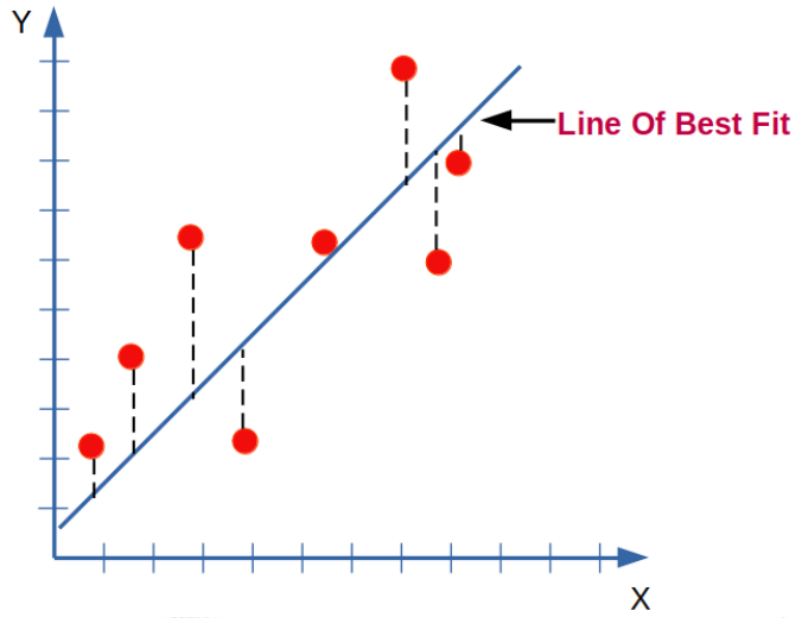


- **y**: The prediction (the estimated price).
- **m**: The slope (how steep the line is; the "weight" given to the feature).
- **x**: The input data (the square footage).
- **b**: The y-intercept (the starting baseline; the "bias").

How It "Learns"

1. This is the most critical concept to explain. If you draw a random line through the data, it won't be perfect. The vertical distance between an actual data point and your predicted line is called a **residual** (or the error).

2. The algorithm calculates the square of all these distances, adds them up to get the total error, and then adjusts the slope and intercept until that total error reaches its absolute minimum.



Example of Linear Regression

Scenario:

You want to predict someone's **weight** based on their **height**.

Data:

Height (inches)	Weight (lbs)
60	115
62	120
64	125
66	130
68	135

Example of Linear Regression

- Linear Regression Concept: Linear regression assumes that there is a straight-line relationship between height (input) and weight (output). The formula for the linear relationship looks like

$$\text{Weight} = m \times \text{Height} + b$$

Where:

- m is the slope of the line (how much weight changes per inch of height),
- b is the y-intercept (the weight when height is zero, which may not make sense in real life but is part of the mathematical model).

Example of Linear Regression

- **Train the Model:** Using the data, the linear regression model finds the best-fitting line that minimizes the error between predicted and actual weights.

- **Fitting the Line:** After training, the model might determine that the best-fitting line is:

$$\text{Weight} = 5 \times \text{Height} - 200$$

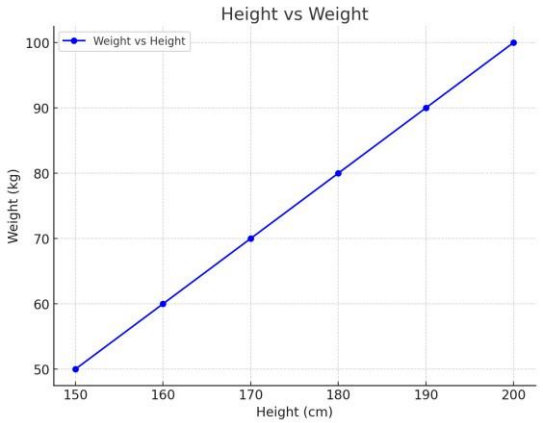
This means for each additional inch in height, the weight increases by 5 lbs, and if the height were 0 inches, the model predicts the weight would be -200 lbs (a mathematical artifact)

Example of Linear Regression

- **Make Predictions:**

  If you have a person who is 70 inches tall, you can predict their weight as:

$$\text{Weight} = 5 \times 70 - 200 = 150 \text{ lbs}$$



Conclusion: In this example, linear regression is used to predict weight based on height. The model learns the relationship between height and weight, then uses that to predict new values for unseen heights.

Implementation: Linear Regression

How it works: Draws a single straight "line of best fit" ($y = mx + b$) through a multi-dimensional cloud of data.

The Learning Process: It calculates the distance between its line and every single data point, then adjusts the line to make those distances as small as possible (Ordinary Least Squares).

Why use it: It is incredibly fast, interpretable, and forms the mathematical foundation of almost all machine learning.

The limitation: Real life is rarely perfectly linear. If house prices curve upwards exponentially based on location, a straight line will struggle.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,
r2_score
```

- Added explicit metric calculations and plain-English print statements
- to translate abstract math into real-world dollars and percentages.

1. Initialize the Model (Creating the blank mathematical equation)

```
lr_model = LinearRegression()
```

2. Train the Model (The 'Learning' phase)

The algorithm adjusts its weights to minimize the error across 80% of our data

```
lr_model.fit(X_train, y_train)
```

3. Predict on the unseen Test data (The Final Exam)

```
lr_predictions = lr_model.predict(X_test)
```

4. Evaluate using human-readable metrics

```
rmse = mean_squared_error(y_test, lr_predictions,
squared=False)
r2 = r2_score(y_test, lr_predictions)
```

Translating the metrics into real-world meaning (Dataset scale is $100k)

```
print(f"RMSE: ${rmse * 100000:,.2f} (On average, our guess
is off by this amount)")
print(f"R2 Score: {r2 * 100:.1f}% (Our straight line
explains this much of the price variance)")
```

```
import joblib
import pandas as pd
```

5. SAVE THE MODEL

```
joblib.dump(lr_model, 'linear_model.pkl')
print("\n model saved as 'linear_model.pkl'")
```

6. LOAD AND INTERACT

```
deployed_lr = joblib.load('linear_model.pkl')
```

```
try:
    med_inc = float(input("Enter Median Income (e.g., 5.0 for $50k):
"))
    house_age = float(input("Enter Average House Age (e.g., 20): "))
    ave_rooms = float(input("Enter Average Rooms (e.g., 5): "))

    # Formatting user input to match the 7 remaining columns in our
training data
    user_data = pd.DataFrame({
        'MedInc': [med_inc], 'HouseAge': [house_age], 'AveRooms':
[ave_rooms],
        'AveBedrms': [1.05], 'AveOccup': [3.0], 'Latitude': [35.6],
'Longitude': [-119.5]
```

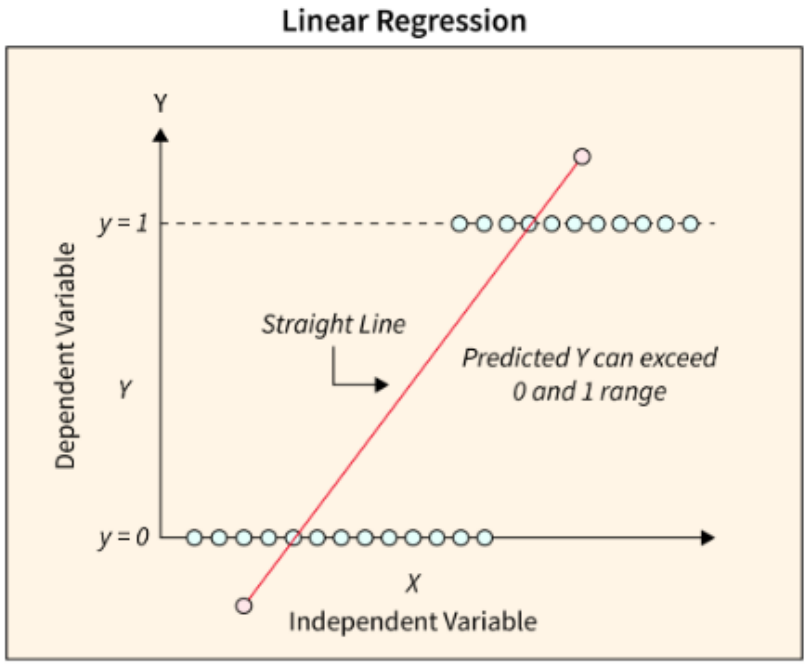
Practical Task

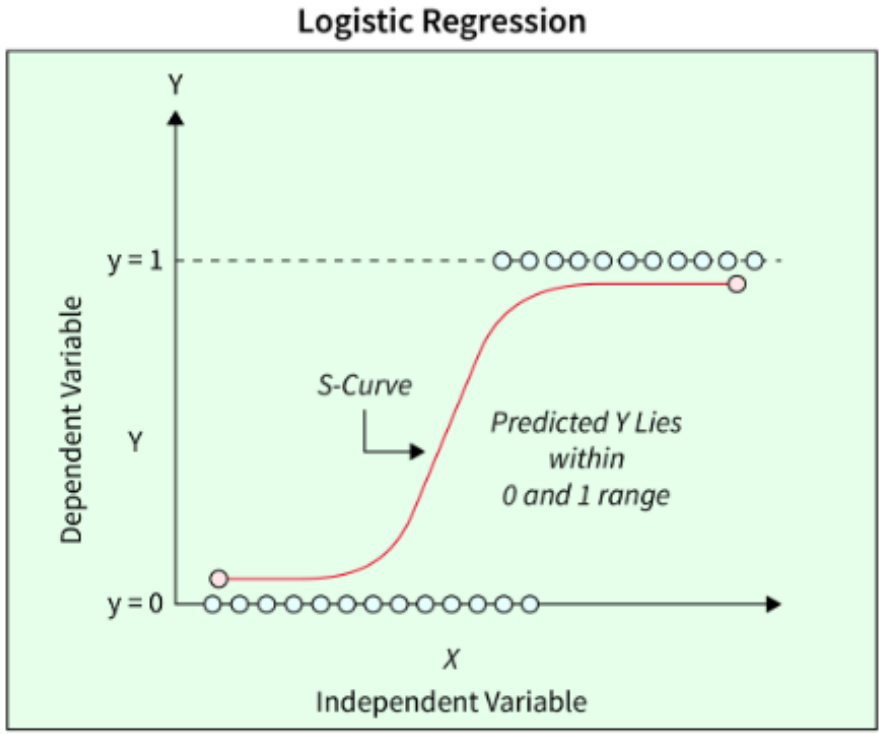
Plot the chart for actual value point and line chart of prediction values by the trained model

- **.fit(),** what is actually *happening?* Imagine you have a whiteboard covered in dots representing house prices. You hold a stiff ruler against the board. You want the ruler to be as close to *every* dot as possible simultaneously. If you move it up, it gets further from the bottom dots. If you tilt it, it gets further from the left dots.
- Linear Regression uses calculus to instantly find the absolute perfect tilt and height for that ruler.
- But ***how do we grade it?*** We look at the **RMSE** (Root Mean Squared Error). In plain English, RMSE tells us our average mistake.

Logistic Regression

Logistic Regression is a Classification algorithm. It is the absolute gold standard in the medical and banking fields for predicting probabilities.





Logistic Regression

The Goal: Predict if a patient has diabetes (1 = Yes, 0 = No) based on their health metrics.

The Problem with Linear Regression: A straight line shoots off into infinity. If we use a straight line to predict a "Yes/No" question, it might output a crazy number like 4.2 or -1.5. What does that even mean?

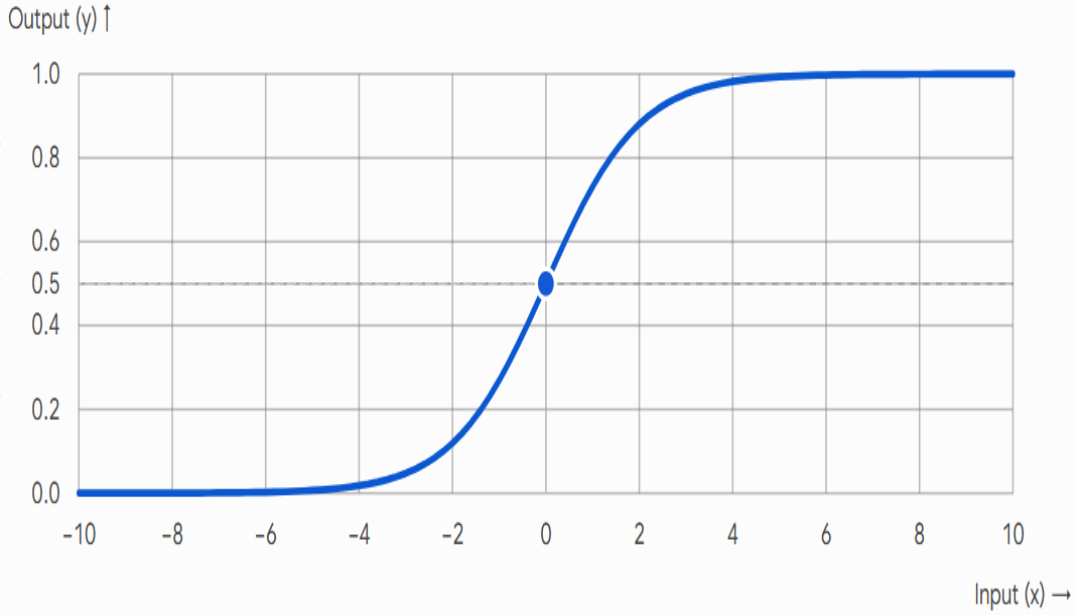
How Logistic Regression Fixes It: It takes that straight line and squishes it into an **"S-Curve"** **Sigmoid Function?**

The Result: It forces the output to be a **Probability** exactly between 0% and 100%. If the probability is > 50%, it predicts 1 (Yes). Otherwise, it predicts 0 (No).

The Sigmoid Formula

$$S(x) = \frac{1}{1 + e^{-x}}$$

The Sigmoid function takes any real number (from negative infinity to positive infinity) and mathematically forces it into a range strictly between **0 and 1**.



Code:

```
X= -10
print(1 / (1 + math.exp(-x)))
```

Or,

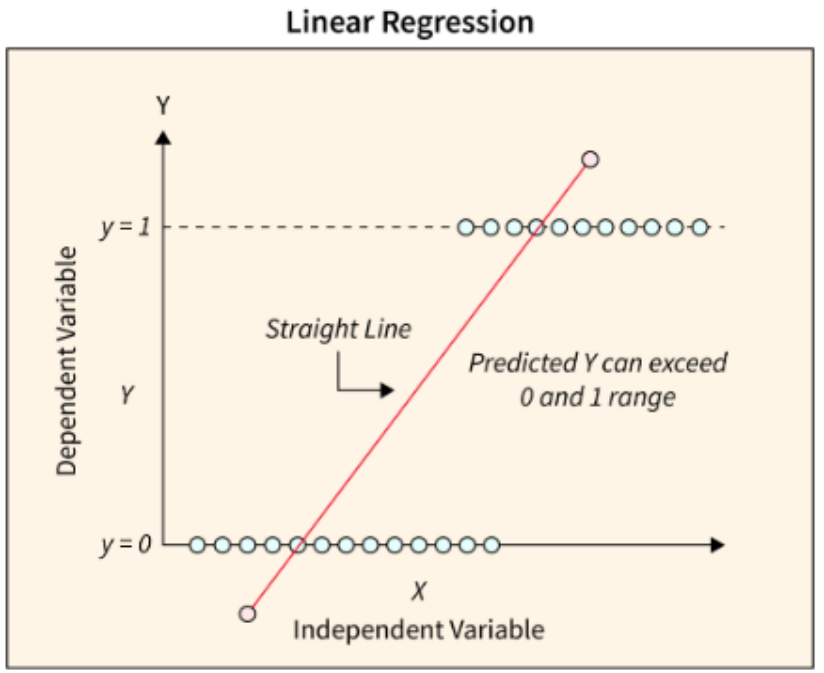
```
import math
X=2.89
print(sigmoid(val))
```

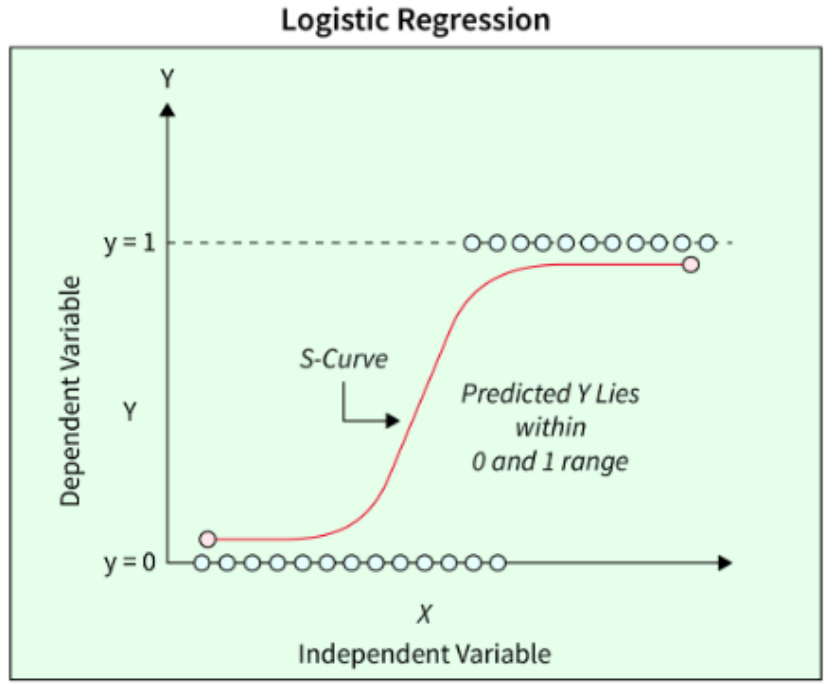
S(x): The output (a probability between 0 and 1).
x: The raw input number (e.g., the output of our linear equation).
e: Euler's number, a mathematical constant approximately equal to 2.71828.

Why Logistic Regression

We cannot use Linear Regression for this. A straight line shoots off into infinity. If a patient is 90 years old with an incredibly high glucose level, a straight line might predict their diabetes status is 2.8. That makes no sense. The answer can only be 0 or 1.

Magic of **Logistic Regression** comes in. It takes that straight line and wraps it in a mathematical corset called a Sigmoid Function. It literally bends the ends of the line so they can never go above 1.0, and never go below 0.0, creating an 'S' shape.





Titanic Survival Prediction using Logistic Regression

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Step 1: Load the online dataset

Seaborn hosts this dataset online,
so it downloads automatically when called.

```
df = sns.load_dataset('titanic')
```

Step 2: Select intuitive features and clean the data

- We will use 'age', 'fare', and 'pclass' (Ticket Class: 1st, 2nd, 3rd)
- Logistic regression can't handle missing data natively, so we drop empty rows.

```
features = ['age', 'fare', 'pclass']
df_clean = df.dropna(subset=features + ['survived'])

X = df_clean[features]
y = df_clean['survived']
```

Step 3: Split the data into Training and Testing sets

We hold back 20% of the passengers to test the model on data it has never seen.

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Step 4: Initialize and Train the Model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Step 5: Test the model

```
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(accuracy )
```

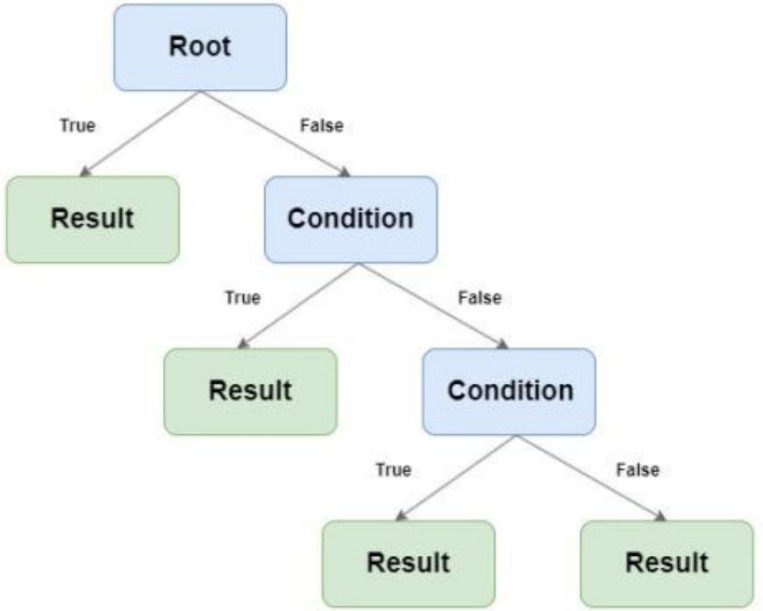
Step 6: Prediction

Let's predict survival for a 20-year-old student with a cheap 3rd class ticket ($15)

```
student_passenger = [[20, 45.0, 2]]
survival_prob = model.predict_proba(student_passenger)[0][1]
print(survival_prob)
```

Decision Tree

- A Decision Tree is used to predict continuous values such as prices or scores using a tree-like structure.
- It splits the data into smaller groups based on simple rules derived from input features, helping reduce prediction errors.
- At the end of each branch, called a leaf node, the model outputs a value, i.e., usually the average of that group.

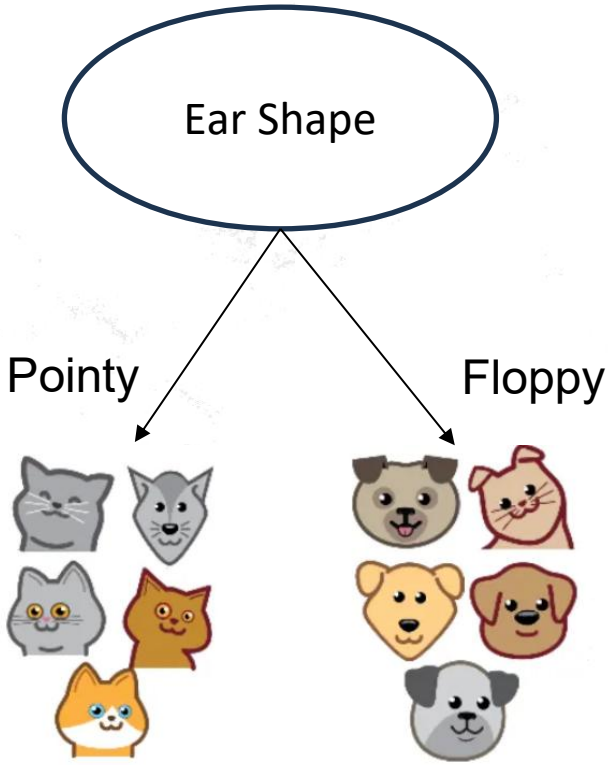


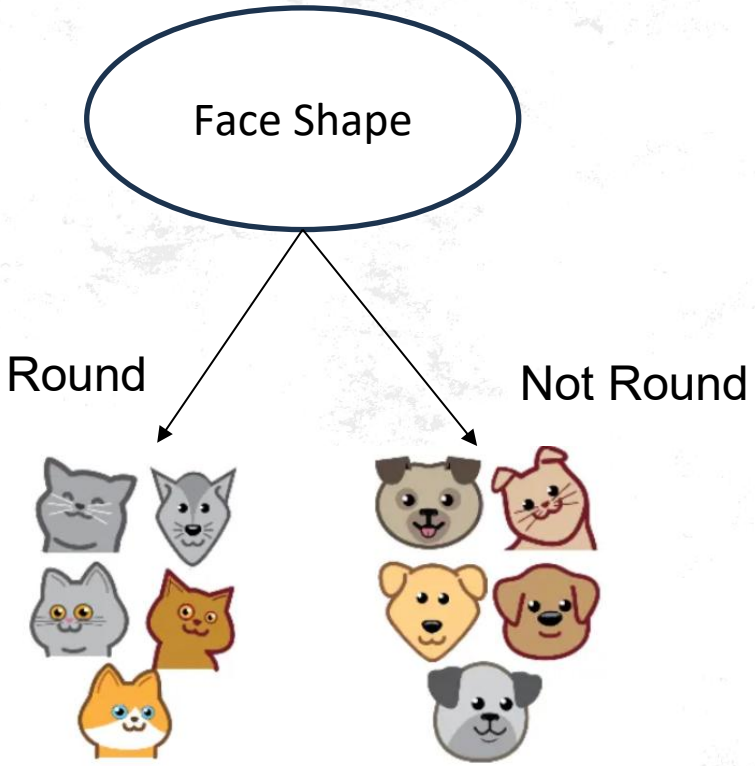
Features Selection

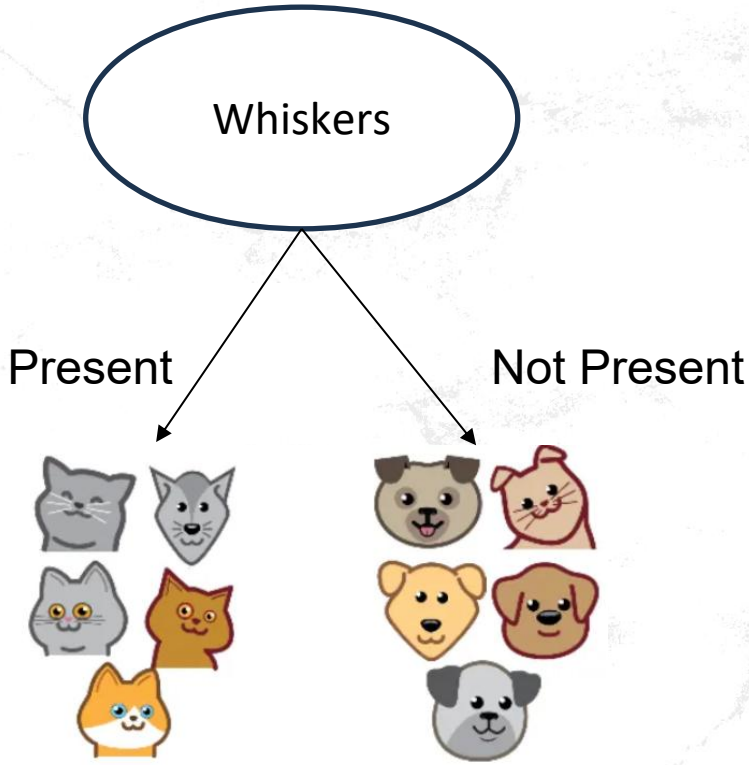
Cat classification example

	Ear shape (x_1)	Face shape (x_2)	Whiskers (x_3)	Cat
	Pointy	Round	Present	1
	Floppy	Not round	Present	1
	Floppy	Round	Absent	0
	Pointy	Not round	Present	0
	Pointy	Round	Present	1
	Pointy	Round	Absent	1
	Floppy	Not round	Absent	0
	Pointy	Round	Absent	1
	Floppy	Round	Absent	0
	Floppy	Round	Absent	0

Dogs and Cats features







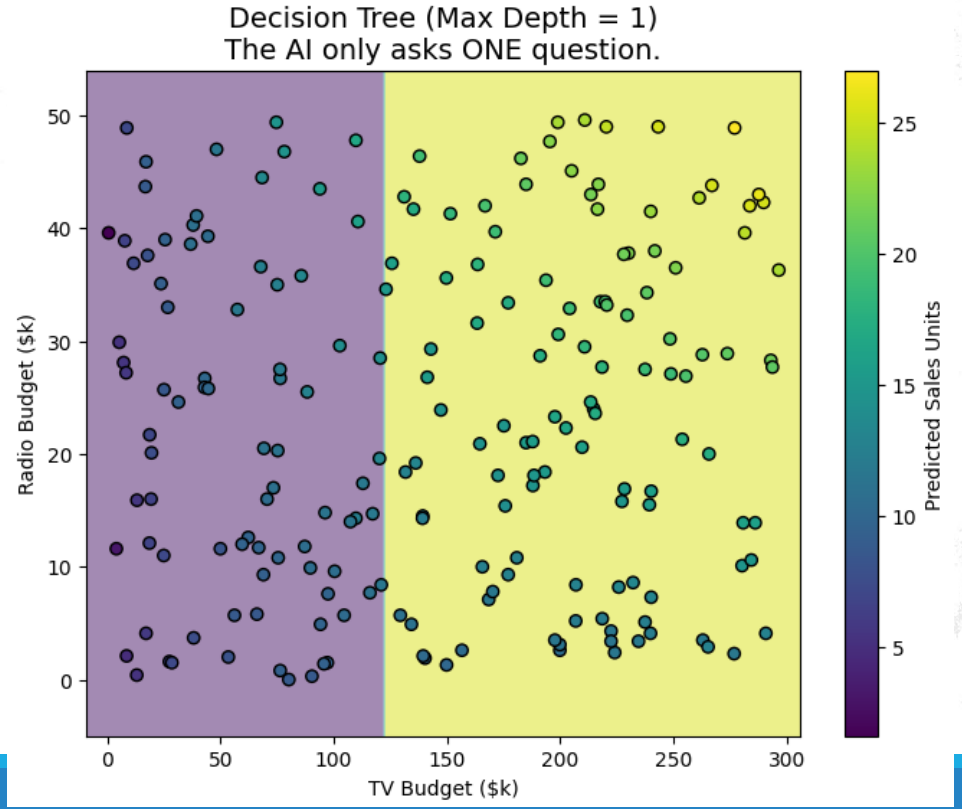
Can we Consider all the features of the dataset??

Budget of TV, Radio and News Paper vs Sales

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9

A Decision Tree draws a series of **straight fences** to chop the graph into little rectangular boxes.

1. The Graph (The Map)
Imagine a 2D map.
The **X-axis** is the TV Budget ($0 to $300k).
The **Y-axis** is the Radio Budget ($0 to $50k).
Every historical advertising campaign is a dot on this map.



2. Drawing the "Fences" (The Splits)

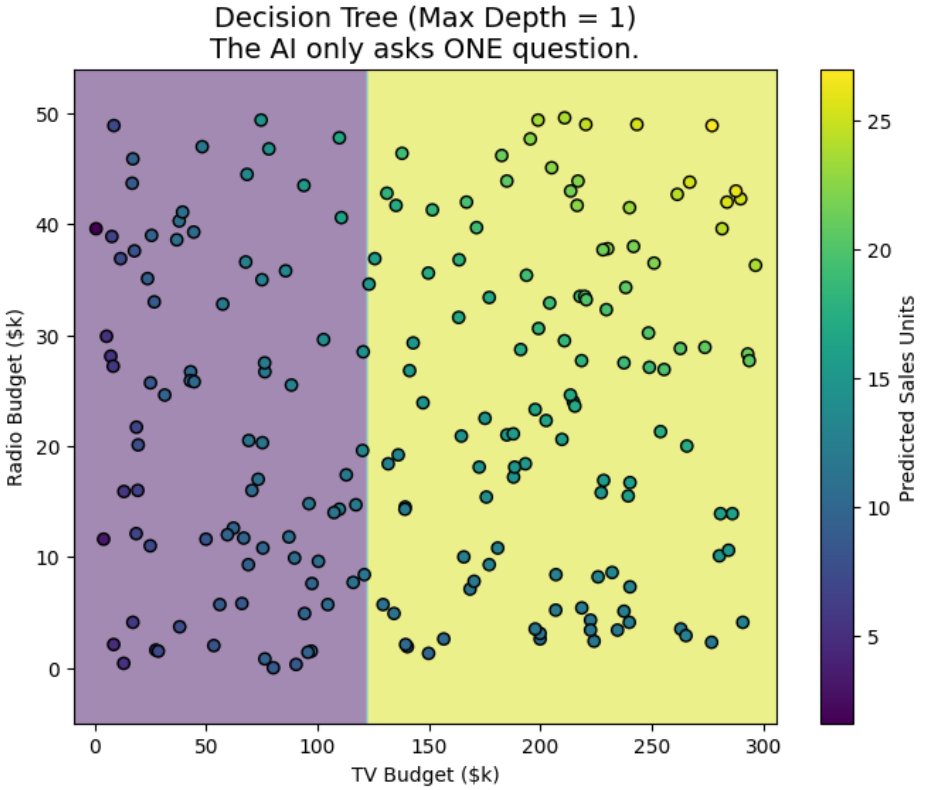
(i)The algorithm looks at the map and says: *"Where can I draw a straight vertical or horizontal line to best separate the high-sales dots from the low-sales dots?"*

It might draw a vertical line at **TV = $150k**.

Now the map is split into a Left Zone and a Right Zone.

(ii) Then it looks at just the Right Zone and says: *"Let's draw a horizontal line at **Radio = $25k**."* *

It keeps drawing these straight lines (up-and-down or left-to-right) until the map is divided into a bunch of rectangular grids.



1. What do the dots represent?

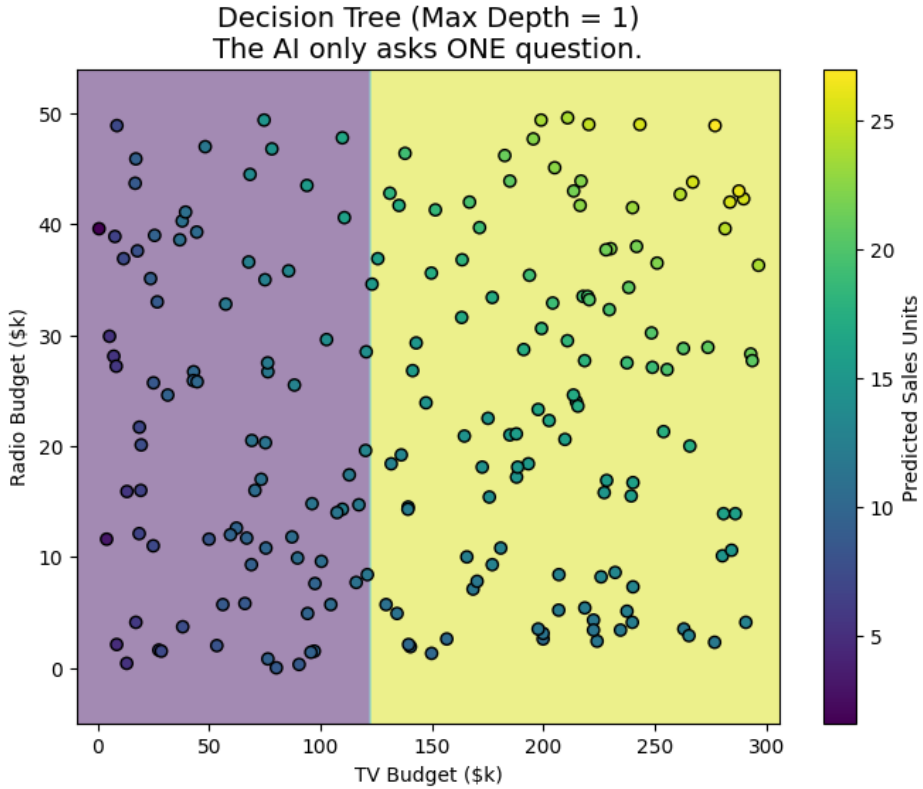
Every single dot is a **real advertising campaign from the past**.

Position (Left/Right): How much money was spent on **TV Ads**. (Dots further to the right spent more on TV).

Position (Up/Down): How much money was spent on **Radio Ads**. (Dots higher up spent more on Radio).

Color of the dot: The color shows the **actual Sales** from that campaign.

Dark purple dots mean terrible sales.
Bright yellow dots mean fantastic sales.



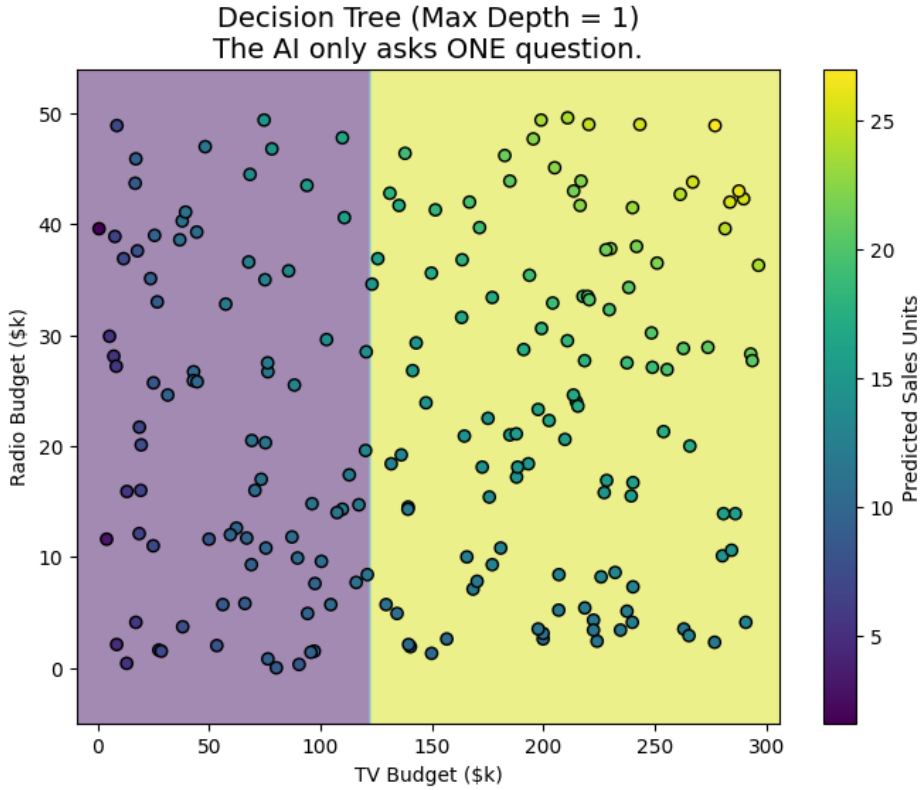
Note: most of the dark purple dots are clustered on the left side (low TV budget), and most of the bright yellow dots are on the right side (high TV budget).

What does "Depth 1" mean?

- "Depth" simply means **"How many questions is the AI allowed to ask?"** Because this is a **Depth 1** tree
- AI was only allowed to ask *one single question* to try and separate the bad sales from the good sales.

"The absolute best single question I can ask is: Did they spend more than $125k on TV?"

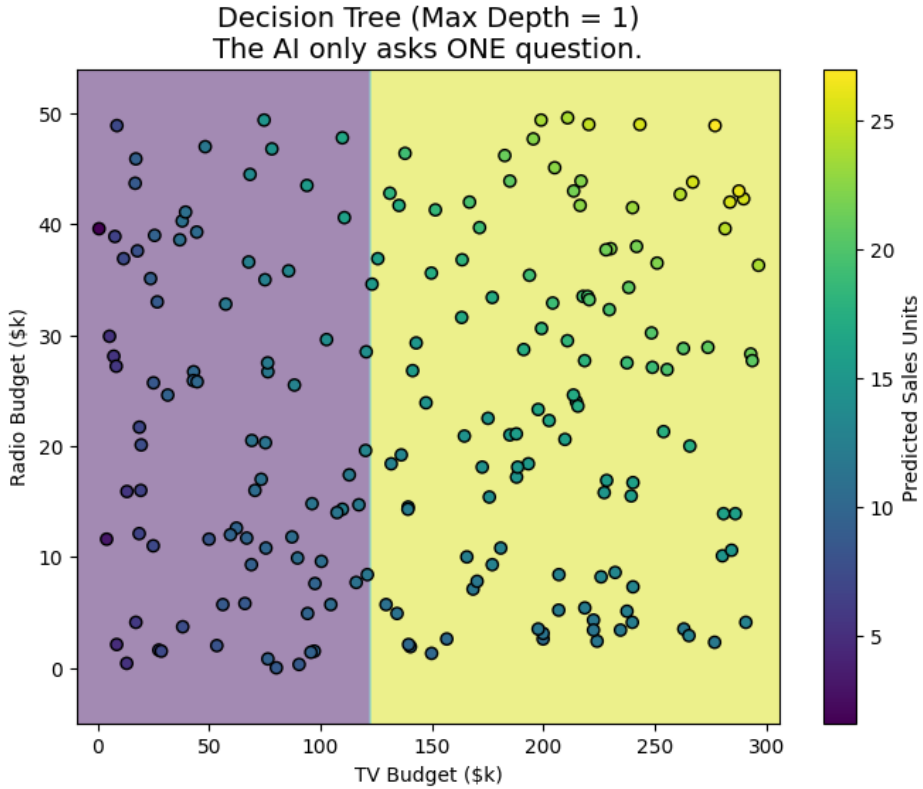
It drew that straight vertical line right down the middle at the 125 mark.



Note:
Depth == No of Question.
Question will be Asked on the variable which have least RMS

What do the Purple and Yellow backgrounds (the canvas) represent?

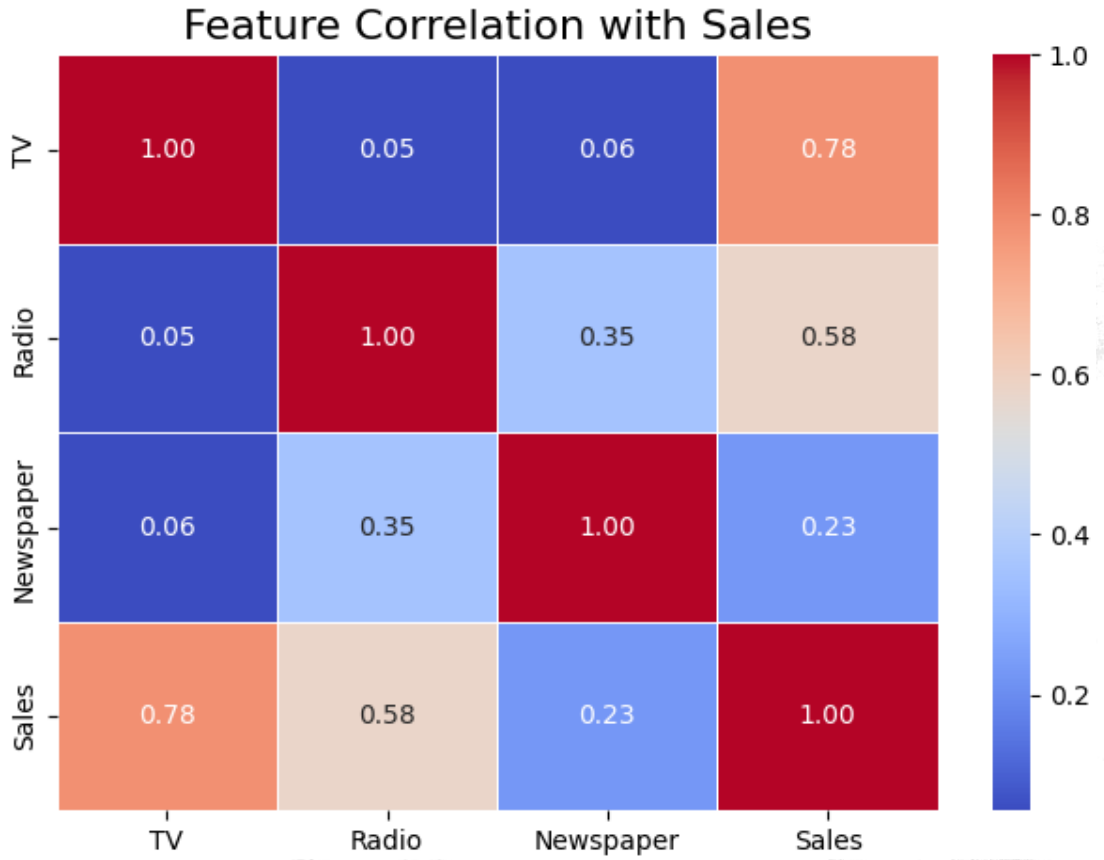
- The background colors represent the **AI's actual predictions for the future**.
- By drawing that line at 125, the AI chopped the graph into two "prediction zones":
- **The Purple Zone:** The AI says, *"If a new company comes to me and says their TV budget is under $125k, I am going to predict they will have **low sales**, because almost all the historical dots in this area are dark purple."*
- **The Yellow Zone:** The AI says, *"If a company tells me their TV budget is over $125k, I will predict **high sales**, because most of the dots over here are bright yellow."*



Question :
What does depth 2 means
and which variable to select?
How many Depths??

why didn't we take Newspaper ?

- TV spending is highly correlated with an increase in Sales.
- Radio spending is moderately correlated with an increase in Sales.
- **Newspaper spending has almost zero actual impact on Sales.**



Question: Can we take 3 variables in decision tree?

https://colab.research.google.com/drive/1wh6G-tczQwBctwqPYI_3p1FTi8D5X3YG?usp=sharing

Underfitting vs Overfitting

Underfitting (Too Simple): The model doesn't learn enough. It draws one or two giant boxes and calls it a day.

E.g: "If TV budget > $125k, sales are high."

Problem: It ignores the Radio completely! It assumes a company that spent $0 on Radio will get the exact same sales as a company that spent $50k.

Overfitting (Too Complex): The model learns *too* much. It memorizes the exact historical data, drawing tiny, hyper-specific boxes around individual outliers.

E.g: "If TV is exactly $214k, and Radio is exactly $39k, sales will be exactly 22 units."

Problem: It memorized random historical noise instead of learning the general trend. It will fail miserably on new, future ad campaigns.

```
import pandas as pd
from sklearn.tree import DecisionTreeRegressor
import joblib

# 1. CREATE A SUPER SIMPLE DATASET
# Students can physically read this and understand the pattern instantly!
data = {
    'Experience_Years': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Certifications':   [0, 0, 1, 1, 2, 2, 3, 3, 4, 4],
    'Salary_k':         [40, 45, 55, 60, 75, 80, 95, 100, 120, 125] # Target
}
df = pd.DataFrame(data)
```

2. PREPROCESS DATA

Separate what we know (X) from what we want to predict (y)

```
X = df[['Experience_Years', 'Certifications']]
y = df['Salary_k']
```

3. TRAIN THE DECISION TREE

```
tree_model = DecisionTreeRegressor(random_state=42)
tree_model.fit(X, y)

print("Decision Tree algorithm has learned the salary rules!")
```

4. SAVE THE MODEL

```
joblib.dump(tree_model, 'salary_tree_model.pkl')
print("Brain saved to disk as 'salary_tree_model.pkl'\n")
```

5. LOAD & INTERACT

```
deployed_tree = joblib.load('salary_tree_model.pkl')

try:
    # Minimal, understandable user input!
    exp = float(input("Enter Years of Experience (e.g., 3): "))
    certs = float(input("Enter Number of Certifications (e.g., 1): "))

    # Package the 2 answers into a Pandas DataFrame
    user_data = pd.DataFrame({
        'Experience_Years': [exp],
        'Certifications': [certs]
    })
```

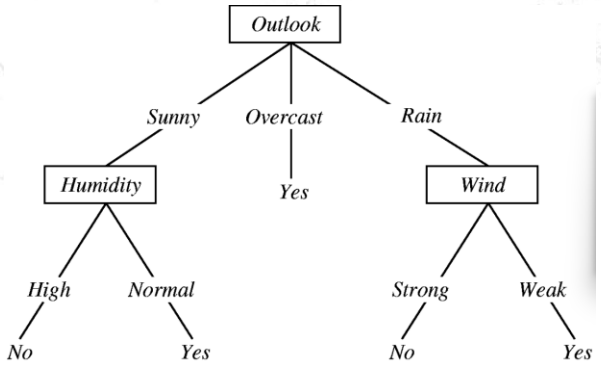
6. MAKE THE PREDICTION

```
    prediction = deployed_tree.predict(user_data)[0]
    print(f"\n AI Prediction: The estimated salary is ${prediction:,.2f}k
per year")


except ValueError:
    print("\nError: Please enter valid numbers only!")
```

Parameters of DecicionTreeRegression()

max_depth	None (Unlimited)	Increases complexity (Overfitting risk)	Reduces complexity (Underfitting risk)
min_samples_split	2	Reduces complexity (Underfitting risk)	Increases complexity (Overfitting risk)
min_samples_leaf	1	Reduces complexity (Underfitting risk)	Increases complexity (Overfitting risk)
max_leaf_nodes	None (Unlimited)	Increases complexity (Overfitting risk)	Reduces complexity (Underfitting risk)

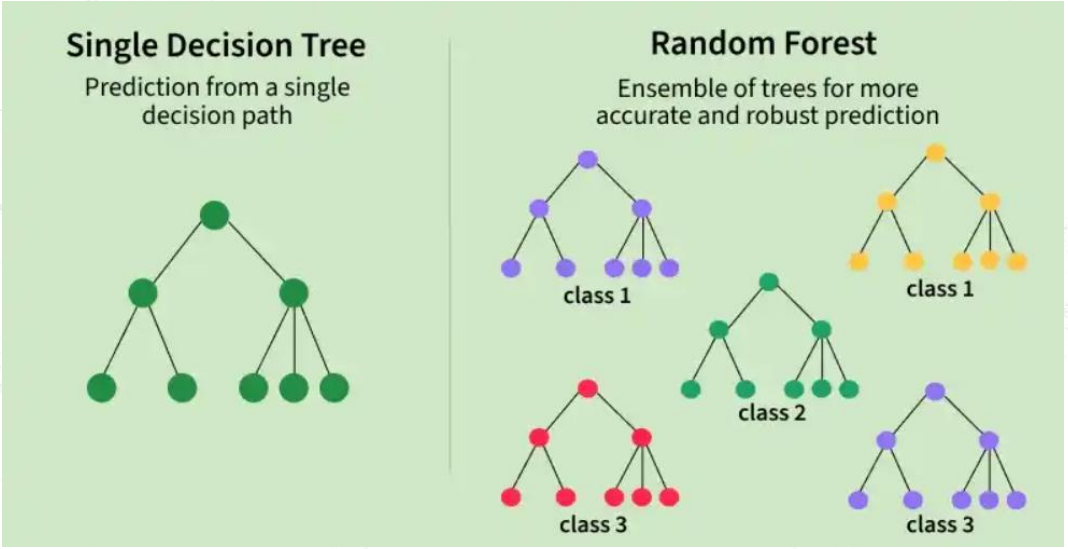


Practical Task

Plot the chart for actual value point and line chart of prediction values by the trained model

Random Forest Regression

- **Random Forest** is an ensemble learning method that combines multiple decision trees to produce more accurate and stable predictions.
- It can be used for both classification and regression tasks, where regression predictions are obtained by averaging the outputs of several trees.



How Random Forest Fixes Decision Tree Weaknesses

Feature	Single Decision Tree	Random Forest
Data Split	Uses the exact same dataset.	Uses random subsets of data per tree (Bagging).
Feature Selection	Looks at all features at every split.	Looks at a random subset of features per split.
Flexibility	Highly sensitive to small data changes.	Highly robust to data changes and outliers.
Computation	Fast to train and easy to visualize.	Slower to train and acts as a "black box".

Weakness of Decision Tree ?

1. **How it gets the Data: Bagging (Row Randomness)**

- Before a single tree is built, the Random Forest creates a unique mini-dataset for it.
- It randomly selects rows from your X_train data *with replacement* meaning the same row can be picked more than once, and some rows are left out completely.

Why? This ensures that every one of your 100 trees is looking at a slightly different version of your patient data.

2. How it chooses a Column: The Random Subspace (Feature Randomness)

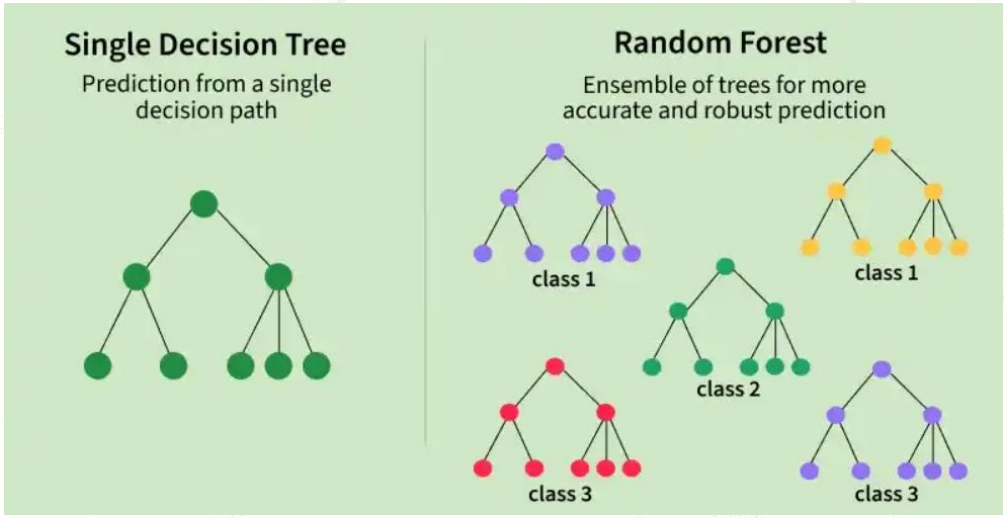
This is the true "secret sauce" of a Random Forest that prevents it from just memorizing the data.
In a standard Decision Tree, at every single fork in the road, the algorithm looks at *all* your columns (Pregnancies, Glucose, Blood Pressure, etc.) to figure out which one splits the Diabetics from the Non-Diabetics the best.

A Random Forest doesn't do that. **At every single split, it is only allowed to look at a random subset of columns.**

The Math: By default, Scikit-Learn uses the square root of the total number of columns. In your code, X has 8 columns **`(df.iloc[:, :8])`**.
Sqrt of 8 columns is around ~ 3

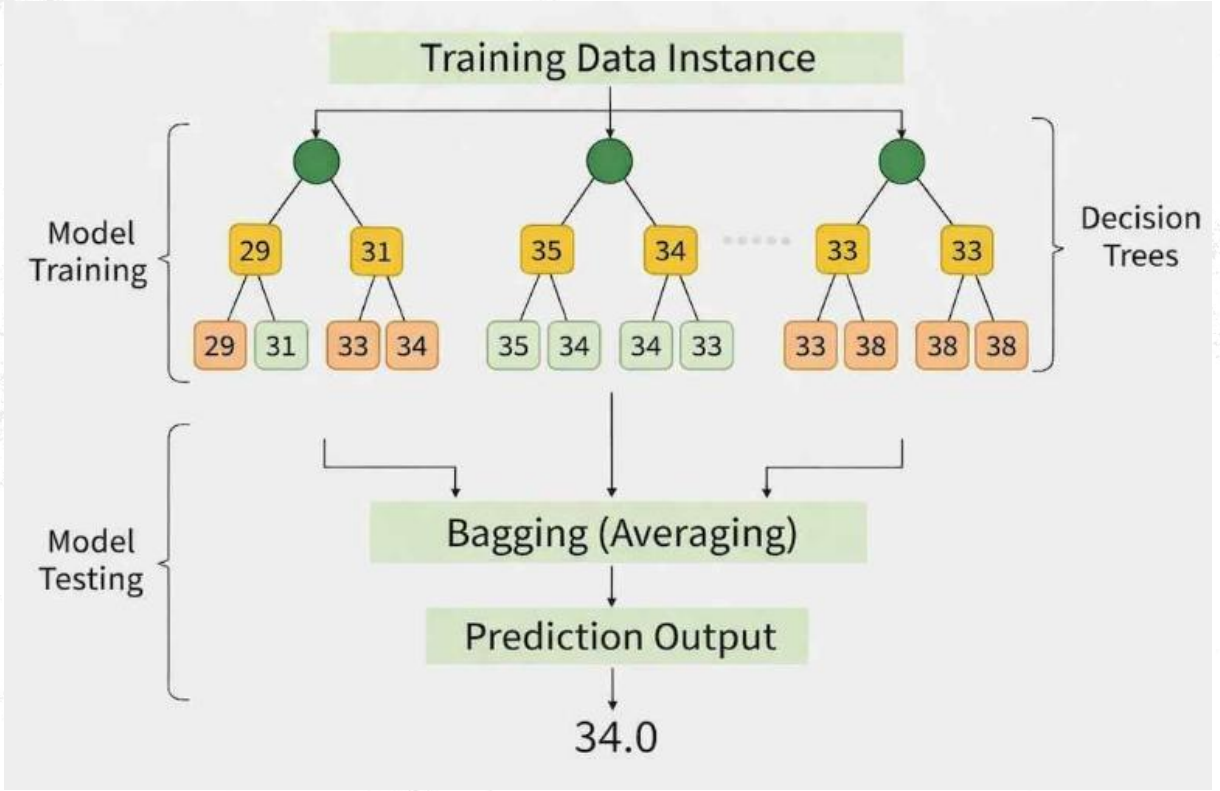
How many questions are asked?

Multiple decision trees: Builds many trees and combines their predictions.
Ensemble approach: Reduces errors compared to a single decision tree by **Combining multiple Model**.
Regression prediction: Produces continuous values by averaging predictions from all trees.



Bagging (Aggregation technique)

- Multiple decision trees are trained on different random subsets of the dataset with replacement to train each tree.
- Each tree uses a random subset of features while splitting nodes.
- Due to this each tree learns slightly different data and features, which increases model diversity.
- The final prediction is obtained by averaging the predictions from all decision trees.



Boosting

- Focuses on improving model accuracy by training models sequentially. Each new model pays more attention to the data points that were misclassified by previous models. Over time, the ensemble becomes better at handling difficult cases.
- Boosting is effective for reducing bias and works well even with weak learners.
- Trains models sequentially
- Gives more weight to misclassified samples
- Combines models using weighted voting
- Reduces bias
- Used in Fraud detection

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
import joblib

1. THE SAME SIMPLE DATASET
data = {
    'Experience_Years': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Certifications':   [0, 0, 1, 1, 2, 2, 3, 3, 4, 4],
    'Salary_k':         [40, 45, 55, 60, 75, 80, 95, 100, 120, 125]
}
df = pd.DataFrame(data)
```

Preprocess: Features (X) vs Target (y)

```
X = df[['Experience_Years', 'Certifications']]
y = df['Salary_k']
```

2. TRAIN THE RANDOM FOREST

 # `n_estimators=100` means we are growing 100 different Decision Trees!

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X, y)
print("Random Forest (100 Trees) has learned the salary rules!")
```

3. SAVE THE MODEL

```
joblib.dump(rf_model, 'salary_rf_model.pkl')
print("Forest saved to disk as 'salary_rf_model.pkl'\n")
```

4. LOAD & INTERACT

deployed_rf = joblib.load('salary_rf_model.pkl')

```
try:
    # Minimal user input
    exp = float(input("Enter Years of Experience (e.g., 3): "))
    certs = float(input("Enter Number of Certifications (e.g., 1): "))

    user_data = pd.DataFrame({
        'Experience_Years': [exp],
        'Certifications': [certs]
    })
```

5. MAKE THE PREDICTION

```
    prediction = deployed_rf.predict(user_data)[0]
    print(f"\nAI Prediction: salary is ${prediction:,.2f}k per year")


except ValueError:
    print("\nError: Please enter valid numbers only!")
```

RandomForestRegressor Uses Bagging or Boosting?

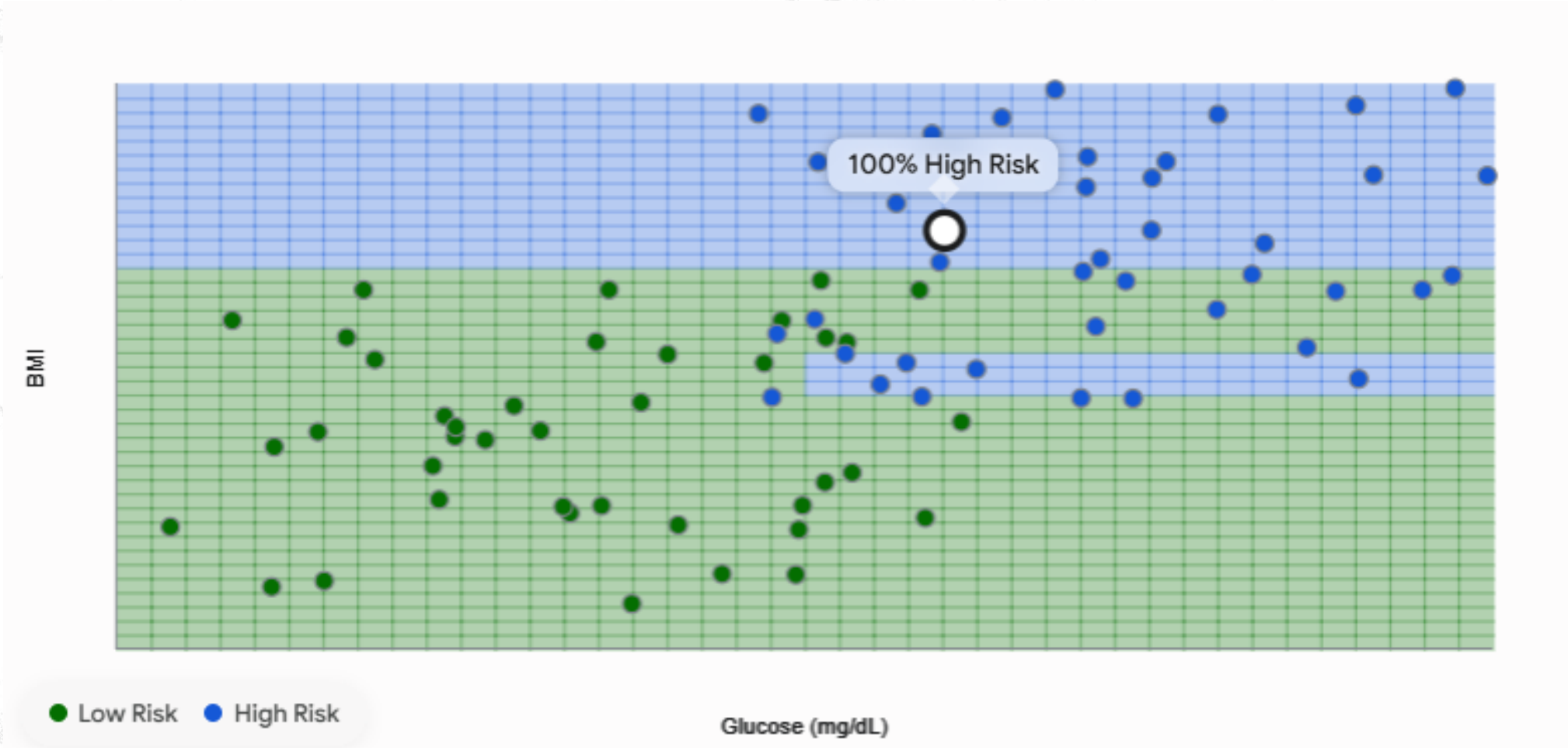
Note: Bagging (Bootstrap Aggregating): Used by Random Forest. The algorithm builds dozens of trees **in parallel** (at the same time). Every tree is completely independent and has no idea the other trees exist. They all vote at the end.

Random Forest on diabetes dataset

The Goal: Diagnose diabetes using a "Wisdom of the Crowd" approach.
The Concept: If you have a complex illness, do you want one doctor, or a board of 100 doctors consulting together?

How it works (Bagging): 1. It builds 100 different Decision Trees. 2. It gives every tree a slightly different subset of patient data to study. 3. It forces each tree to look at different health metrics (some look at BMI + Age, others look at Glucose + Insulin).
The Prediction (Majority Vote): When a new patient arrives, all 100 trees vote. If 85 trees say "Diabetes" and 15 say "Healthy", the forest outputs an 85% probability.

Forest made up of Trees?



XGBoost

Bagging (Random Forest): Like asking 100 independent doctors for a diagnosis at the exact same time and taking a majority vote. They don't talk to each other.

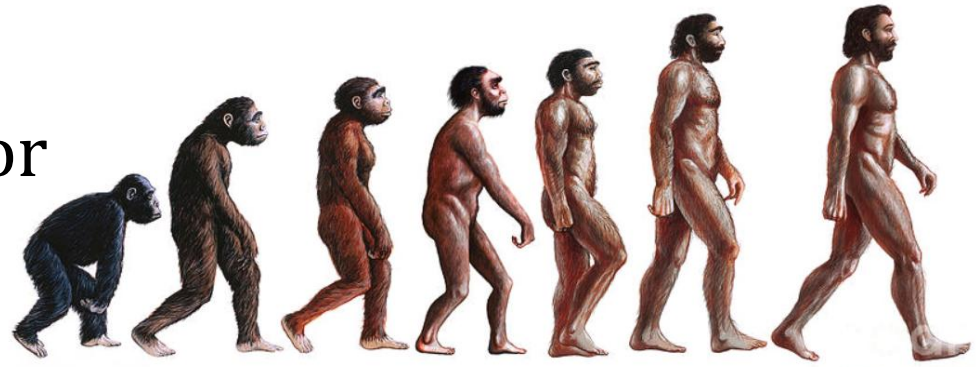
Boosting (**XGBoost**): Like a sequential relay race.

Doctor 1: makes a diagnosis.

Doctor 2: looks only at the patients Doctor 1 misdiagnosed and tries to fix those specific errors.

Doctor 3: looks only at the mistakes of Doctor 2.

The model gets progressively smarter by obsessing over its own failures.



$$\text{Residual} = \text{Actual Value} - \text{Predicted Value}$$

Tree 1: Makes a base prediction.
Tree 2: Trains entirely on the Residuals of Tree 1 to fix its mistakes.
Update Rule: We calculate the new prediction using a learning_rate(n) to prevent over-correcting:

$$\text{New Prediction} = \text{Old Prediction} + (\eta \times \text{Current Tree Prediction})$$

Hyperparameters in XGB model

Confidence of the Correction: The Greek letter **Eta (η)**which is the mathematical symbol used to represent the **Learning Rate**. It tells the deflection in prediction from the previous tree between **0 to 1**

For example: Lets take Doctors.

If η = 1.0 (High Ego): Doctor 2 says, *"Doctor 1 was completely wrong. Ignore them, my diagnosis is the absolute truth."* (This leads to wild swings and overfitting).

If η = 0.3 (Humble): Doctor #2 says, *"Doctor 1 was slightly off. Let's just adjust their original diagnosis by 30% in my direction."*

`n_estimators`: No of trees (NO of random split of a dataset)

`learning_rate`: Rate of change or how slow we have to move ahead in prediction process. It slows down the learning process.

`max_depth`: No of question a tree allowed to ask

`random_state`: It locks the randomness of stochastic algorithms

```
xgb_model = xgb.XGBClassifier(
    n_estimators=10,
    max_depth=3,
    learning_rate=0.3,
    random_state=42
)
```

LogLoss in XGBoost model

The AI is Right & Confident (Prediction: 99%): The AI says, *"I am absolutely certain they are diabetic."* Because it was correct and highly confident, Log Loss gives it a penalty of almost **0**.

The AI is Right but Guessing (Prediction: 51%): The AI says, *"Uhh, I think they are diabetic, but I'm basically flipping a coin."* Even though it technically guessed the right category, Log Loss gives it a **moderate penalty** because it was unsure.

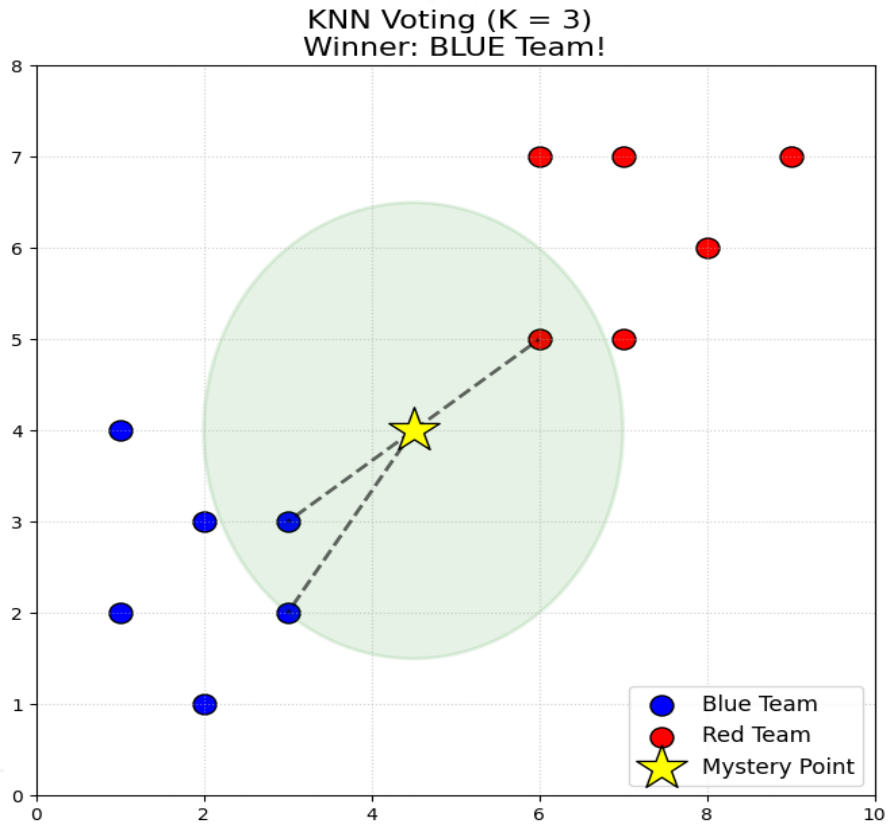
The AI is Confidently WRONG (Prediction: 1%): The AI says, *"I am absolutely certain this patient is completely healthy."* This is the worst-case scenario. The AI is not just wrong; it is a confident liar. Log Loss will hit the model with a **massive, exponentially huge penalty**.

```
xgb_model.fit(X_train, Y_train, eval_set=eval_set, verbose=True)
```

K-Nearest Neighbors (KNN)

Logistic Regression used math equations. Decision Trees used logical flowcharts. **K-Nearest Neighbors (KNN)** uses physical distance. It operates on a very simple human concept: *"Show me your friends, and I will tell you who you are."*

1. It looks at our setting: `K=3`.
2. It takes a compass, puts the needle on the star, and draws a green circle that gets wider and wider until it captures exactly 3 dots.
3. Look at the black dashed lines! Those are the 3 neighbors.
4. The AI takes a vote. In this circle, how many are Blue? How many are Red? Whichever color has the most votes wins, and the yellow star joins that team!"



The Concept: "Birds of a feather flock together." Patients with similar health metrics likely have similar health outcomes.
How it works: It doesn't learn a math equation. It literally memorizes the entire dataset! When a new patient arrives, it calculates the distance to every historical patient, finds the "K" closest ones (e.g., K=5), and lets them vote.
The Crucial Step (Scaling): Because it uses physical distance, large numbers (Glucose = 150) will completely overpower small numbers (Pedigree = 0.5). We must use a StandardScaler to shrink all columns to the same proportion!

Model Evaluation

Let's talk about the most important phase of a Data Scientist's job: **Model Evaluation**. Building a model is easy; proving that it actually works in the real world is the hard part.

Why do we Train/Test Split?

Imagine giving a student a practice math test on Monday, and then giving them the *exact same test* on Friday for their final grade. If they get a 100%, did they actually learn math, or did they just memorize the answers?

If we train our AI on 100% of our data, it will just memorize the spreadsheet (Overfitting). To see if it actually learned the *rules*, we have to hide 20% of the data in a vault, train the AI on the 80%, and then test it on the 20% it has never seen before.

```
# 2. Train/Test Split
X_train_C, X_test_C, Y_train_C, Y_test_C = train_test_split(X_clf, Y_clf, test_size=0.2,
random_state=42)


# 3. Train and Predict
clf_model = LogisticRegression(max_iter=1000)
clf_model.fit(X_train_C, Y_train_C)
clf_predictions = clf_model.predict(X_test_C)
```

Grading Regression Models

If the actual salary is $60,000 and the AI predicts $59,999, it is technically "wrong," but practically, it's a fantastic prediction!

MSE / RMSE (Mean Squared Error / Root Mean Squared Error):
This tells you exactly how far off your predictions are, on average, in real-world units.

What it tells you: If your RMSE is 5, it means your AI's predictions are usually off by about $5,000.
If this number is huge, your model is underfitting (guessing blindly).

```
reg_model = LinearRegression()
reg_model.fit(X_train_R, Y_train_R)
reg_predictions = reg_model.predict(X_test_R)

# 4. The Report Card (Regression)
rmse = mean_squared_error(Y_test_R,
reg_predictions, squared=False)
```

R2 Score

R^2 Score (Coefficient of Determination): This grades the model on a scale from 0.0 to 1.0. It compares your AI to a "dumb" model that just guesses the average every time.

What it tells you: An R^2 of 0.85 means your AI successfully explained 85% of the patterns in the data. An R^2 of 0.0 means your AI is literally no better than just guessing the average.

```
r2 = r2_score(Y_test_R, reg_predictions)
```

Grading Classification Models

When predicting "Yes/No" or "Sick/Healthy", overall Accuracy is often a trap.
Imagine a rare disease that only 1 in 100 people have.
I could write a "dumb" AI that simply predicts "Healthy" for every single patient.
It would be 99% accurate, but it would be completely useless because it missed the 1 sick person!

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

We break Accuracy down using a **Confusion Matrix** (True Positives, False Positives, True Negatives, False Negatives):

Precision

Precision: *"When the AI cries wolf, is there actually a wolf?"*

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

```
# 4. The Report Card (Classification)
acc = accuracy_score(Y_test_C, clf_predictions)
prec = precision_score(Y_test_C, clf_predictions)
rec = recall_score(Y_test_C, clf_predictions)
f1 = f1_score(Y_test_C, clf_predictions)
```

The Warning Sign: If Precision is low, your AI is hyper-paranoid. It is diagnosing healthy people as sick (False Positives).

Recall (Sensitivity):

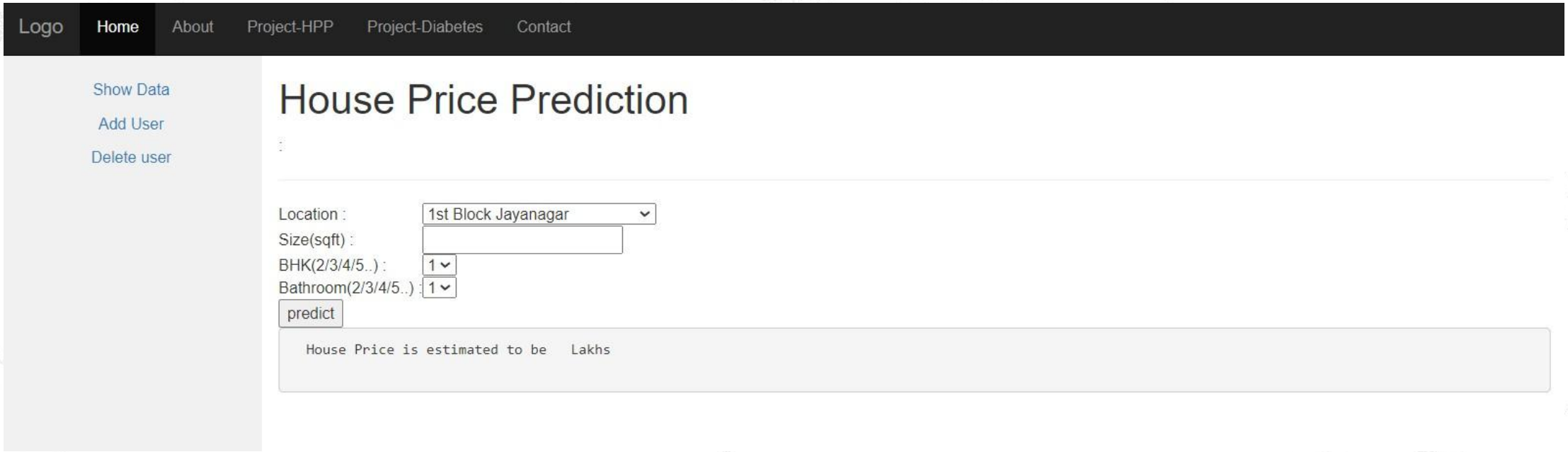
"Did the AI find all the wolves?"

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

The Warning Sign: If Recall is low, your AI is dangerously oblivious. It is telling sick patients that they are perfectly healthy (False Negatives). In medicine, a low Recall is catastrophic.

F1-Score: The mathematical harmonic mean of Precision and Recall. It gives you one single score that proves the model is balanced.

House Price Prediction



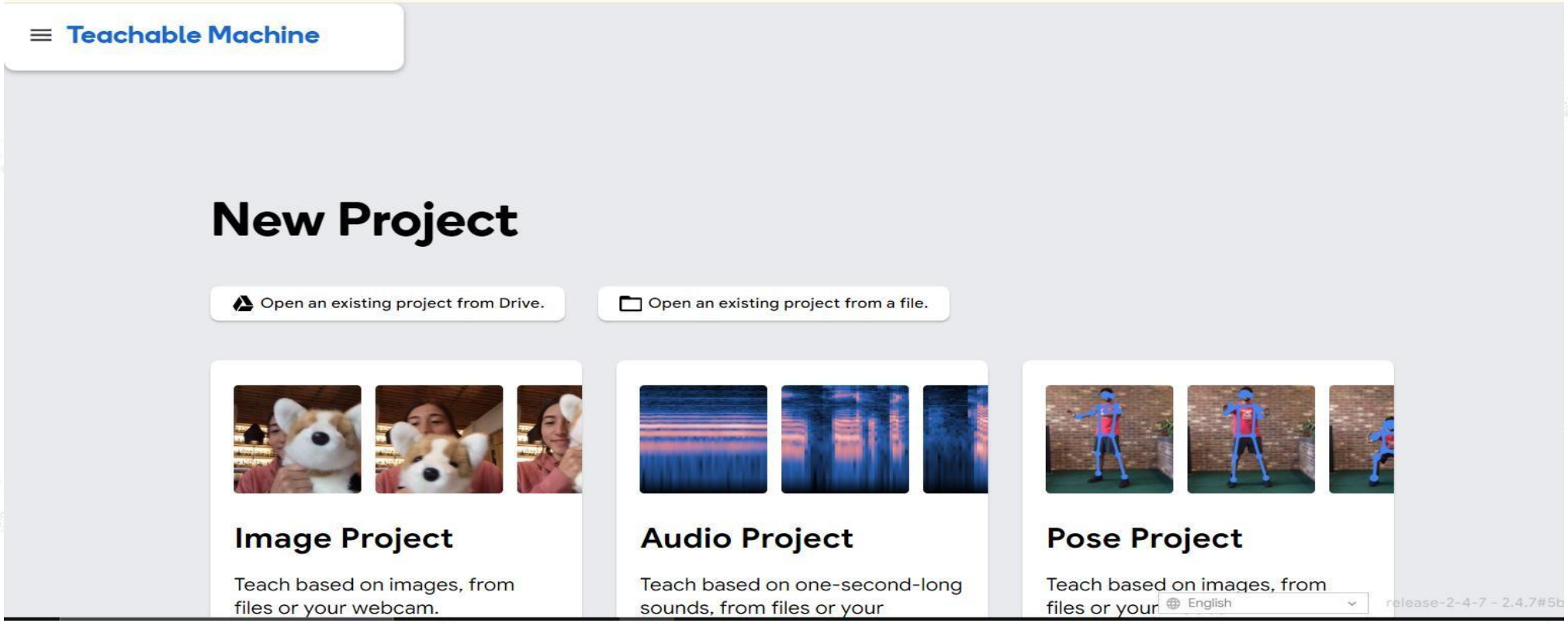
https://house-price-predication.glitch.me/project click here

Classification Vs Regression

Task	Classification	Regression
What it does	Sorts data into categories	Predicts a number
Example	Is the email "spam" or "not spam"?	What will the house price be?
Real-Life Use	Recognizing animals in photos	Forecasting the weather temperature

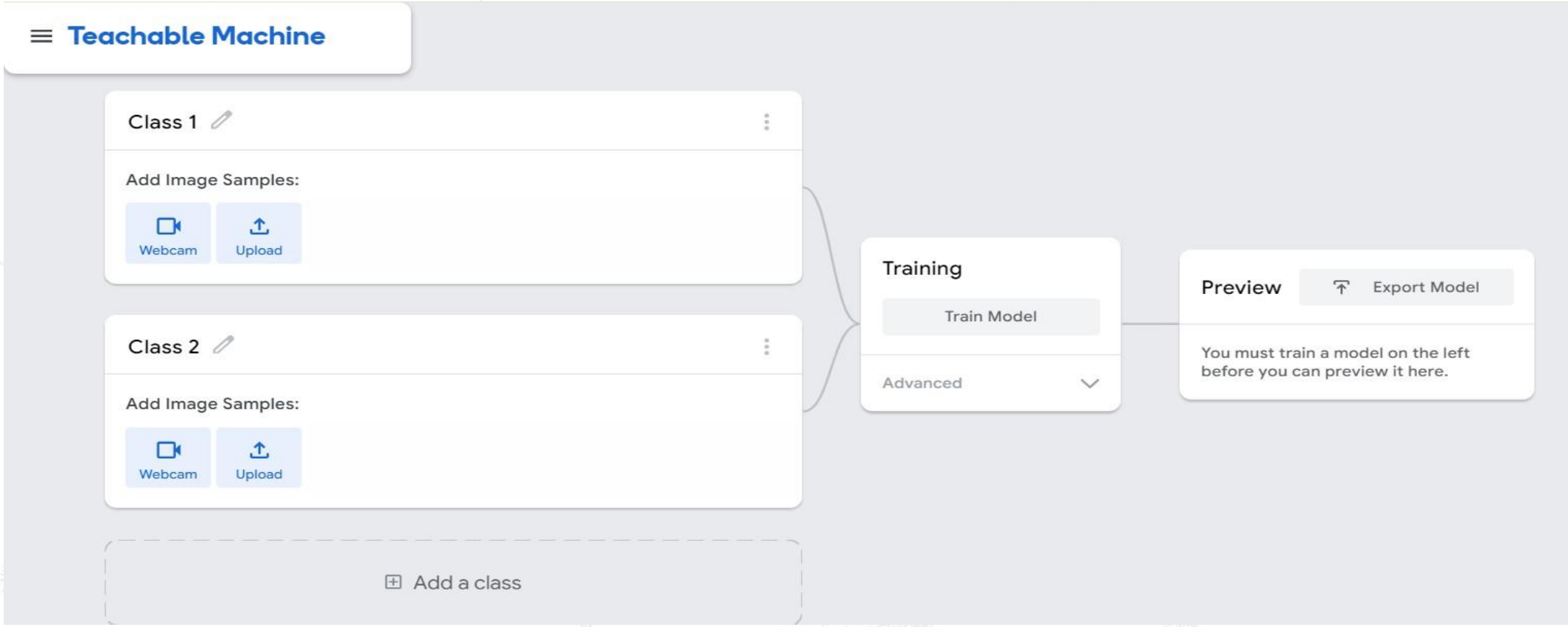
Example of Machine Learning

- Google's **Teachable Machine**
  https://teachablemachine.withgoogle.com/train



Example of Machine Learning

Image Project



Unsupervised learning

" letting the dataset speak for itself "

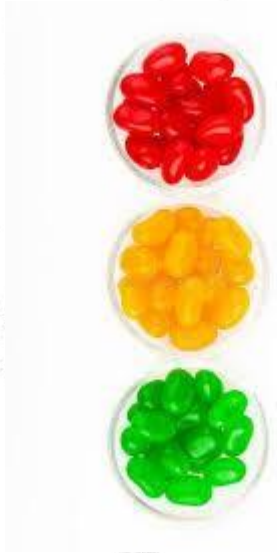
- Unsupervised learning is like letting the machine explore on its own without giving it any answers (no labels). It looks at the data and tries to find patterns or groups. Think of it as solving a puzzle without knowing what the final picture looks like.

- How Does It Work?

  The computer is given data and told:"Look at the information and figure out what things are similar."It groups similar items together or finds hidden patterns.

Activity

- Imagine you have a jar of mixed candies of different colors.
- You didn’t label the candies (no one told you which is which).
- You want to group them by color (red, yellow, green).
- You observe and group the candies based on their similarities.



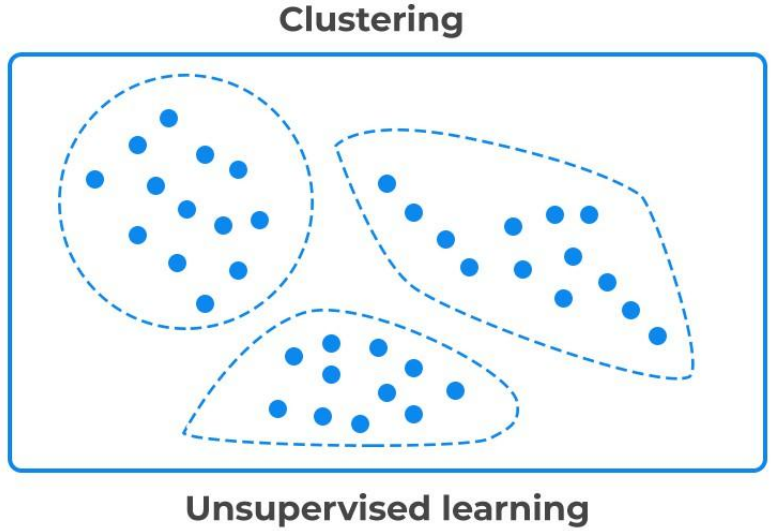
Application of Unsupervised learning

- Recommendation systems
- Customer segmentation
- Document and Text Analysis
- Image and Video Processing
- Anomaly Detection
- Behavioural Analysis
- Stock Market Analysis
- *and many more….*

Unsupervised learning Clustering

Clustering is a technique in machine learning where we group similar things together.

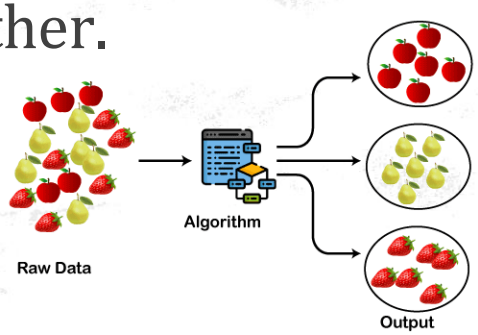
Imagine you have a bunch of different types of fruits, like apples, bananas, and oranges. Clustering is like sorting them into separate baskets based on how similar they are. So, all apples go into one basket, bananas into another, and oranges into another.



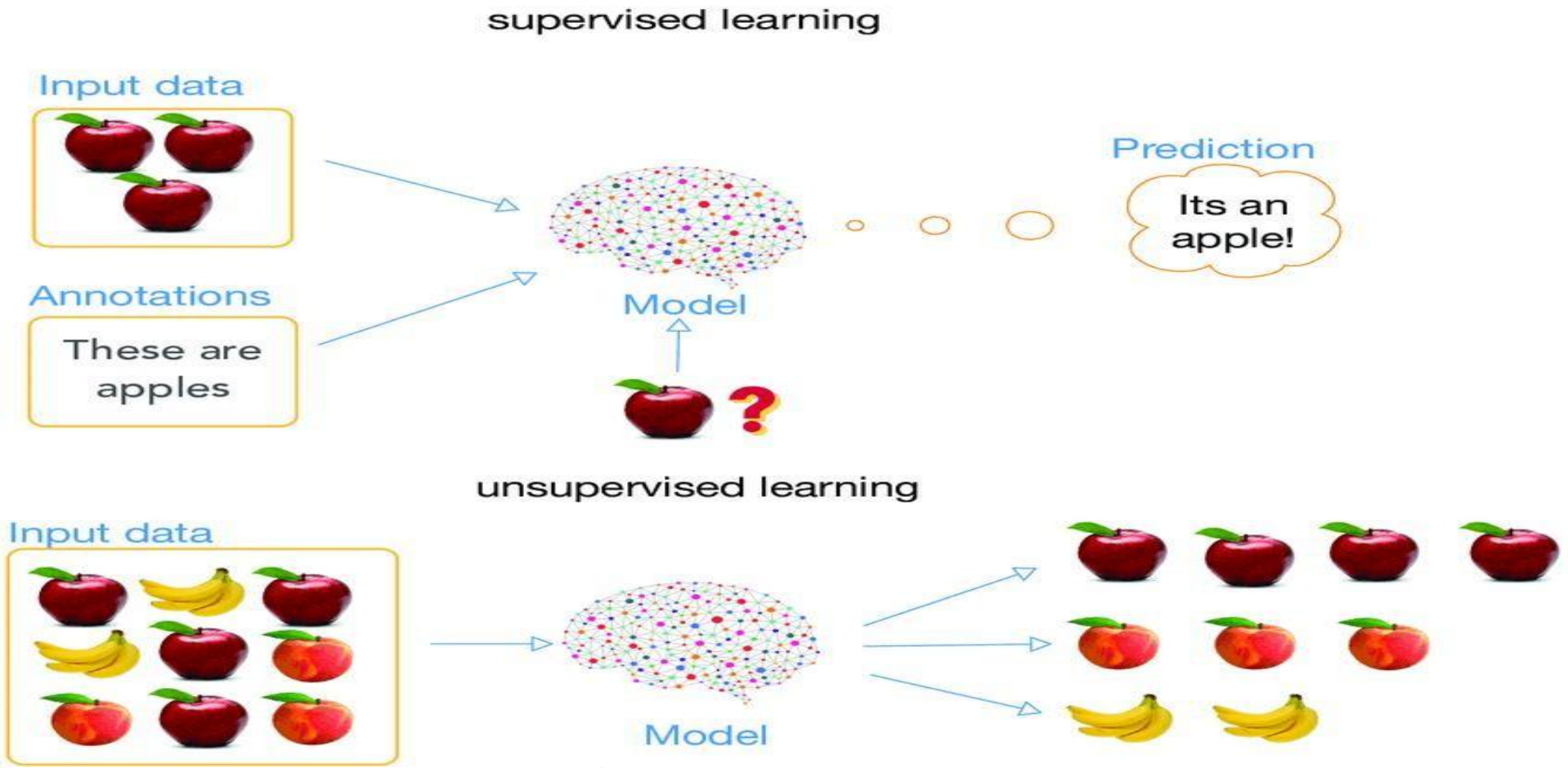
Clustering :How it works ?

Here's how it works in simple steps:

1. **Input Data:** We give the machine data points (like the features of each fruit or customer).
2. **Find Similarities:** The machine looks for patterns and figures out which data points are similar to each other.
3. **Group the Data:** Based on these similarities, it puts the data points into different groups (or clusters).
4. **Results:** Each group contains items that are similar to each other.



Supervised vs Unsupervised learning



Supervised vs Unsupervised learning

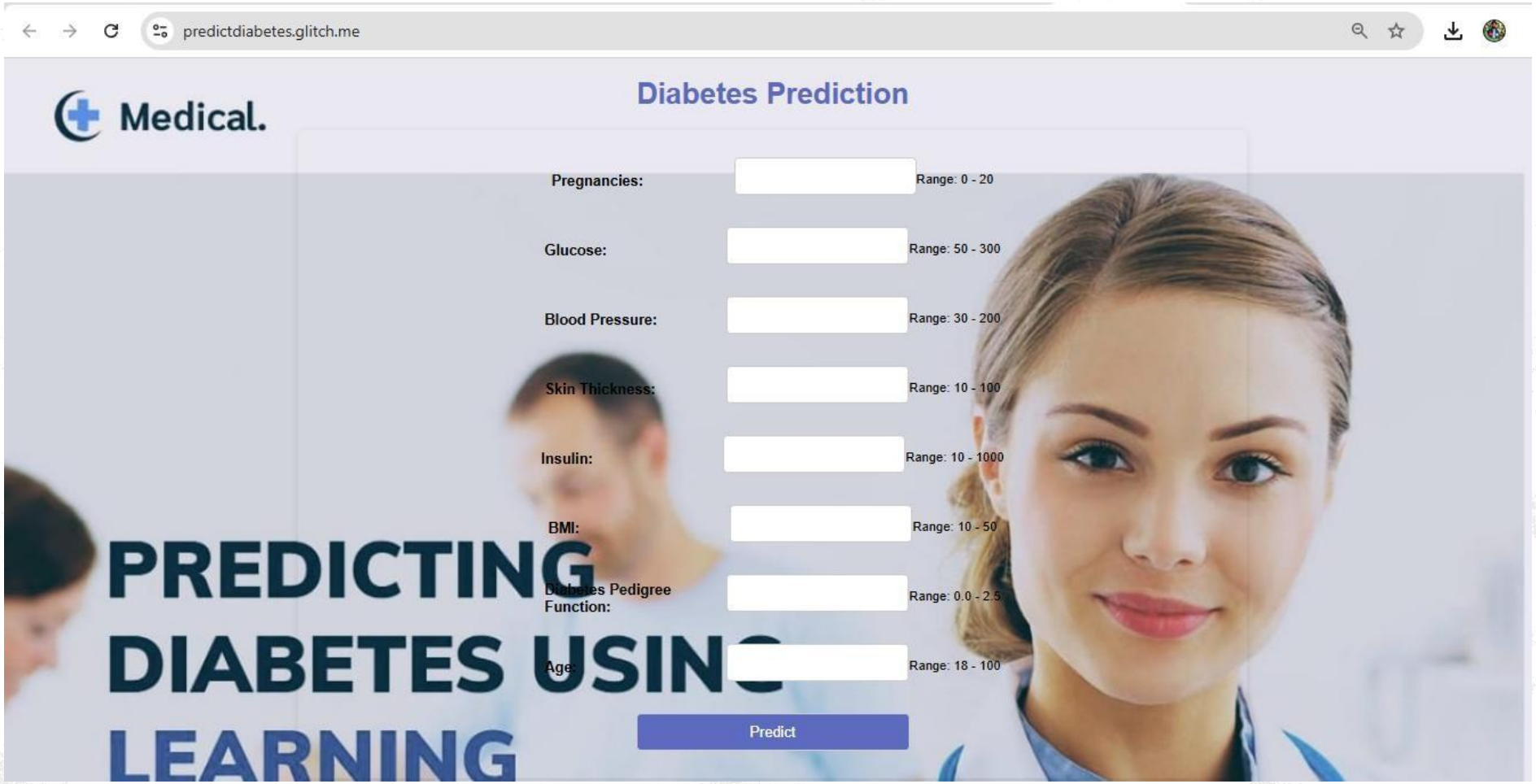
Supervised learning	Unsupervised learning
Input data is labeled	Input data is unlabeled
Has a feedback mechanism	Has no feedback mechanism
Data is classified based on the training dataset	Assigns properties of given data to classify it
Divided into Regression & Classification	Divided into Clustering & Association
Used for prediction	Used for analysis
Algorithms include: decision trees, logistic regressions, support vector machine	Algorithms include: k-means clustering, hierarchical clustering, apriori algorithm
A known number of classes	A unknown number of classes

Diabetes Prediction

	A	B	C	D	E	F	G	H	I
1	Pregnancie	Glucose	BloodPres	SkinThickn	Insulin	BMI	DiabetesP	Age	Outcome
2	6	148	72	35	0	33.6	0.627	50	1
3	1	85	66	29	0	26.6	0.351	31	0
4	8	183	64	0	0	23.3	0.672	32	1
5	1	89	66	23	94	28.1	0.167	21	0
6	0	137	40	35	168	43.1	2.288	33	1
7	5	116	74	0	0	25.6	0.201	30	0
8	3	78	50	32	88	31	0.248	26	1
9	10	115	0	0	0	35.3	0.134	29	0

https://raw.githubusercontent.com/sarwansingh/Python/master/ClassExamples/data/diabetes1.csv

Diabetes Prediction



https://predictdiabetes.glitch.me/

Thank You